

第20章 面向对象程序设计

20.1 设计的一般过程

在面向对象程序设计中，通常遵循以下步骤：

- (1) 建立模型
- (2) 细化模型
- (3) 抽象与表示类型
- (4) 分离并封装变化
- (5) 通过子类型化适应变化
- (6) 迭代上述过程

建立模型是指利用依赖、关联、组合、聚合等关系构建高层次的关系模型。此过程中需分析问题域，依据领域知识和经验，根据事物的公共行为能力，用水平关系将各事物联系起来；细化模型是针对模型的粒度进一步细化，通过降低模型的抽象层次，使其更符合设计需求；类型的抽象与表示是将模型中的各部分用类来表示，这涉及类的拆分与合并、行为与数据的表示及组织等；分离并封装变化是指将类中的稳定部分和变化部分分离，并分别封装成独立的类。变化部分可以封装成一个或多个变化类，再用水平关系将稳定类与各变化类联系起来；最后，通过子类型化适应变化，即针对变化的类，允许使用面向对象的动态多态性扩展各变化类，也就是利用公有继承和虚机制来适应各种变化。

在上述过程中，这里主要关注类型抽象与表示、分离并封装变化、以及通过子类型化适应变化。虽然编程语言的熟练程度和编程技巧对这些步骤有一定影响，但真正的设计挑战在于如何有效地区分稳定和变化部分，并将变化封装起来。具体来说：

- 设计的核心是构建一个由类或对象间水平关系组成的“网”。这张“网”中的结点代表抽象出的稳定类或变化类。只有当需要适应特定变化时，才应考虑使用继承。因此，设计过程首先应专注于建立这种水平关系网络，这不仅是设计的重点和难点，也是提高类复用性、体现高水平设计能力的关键所在。
- 在这个网络中，结点分为两类：稳定的类和变化的类。值得注意的是，“稳定是相对的，而变化是绝对的”。确定哪些部分是稳定的，哪些是变化的，通常需要仔细分析软件需求(如变化频率和必要性)以及条件限制(如人员、技术、预算等)。这一过程实际上是一个权衡和妥协的过程，因为实践中不可能要求系统适应所有未来的变化——那样就没有稳定部分了，

也失去了面向对象设计的价值。此外，讨论稳定性和变化性时，必须限定在特定的问题域内。识别潜在的变化是设计阶段的重要任务，但针对哪些变化进行设计并没有标准答案，只能根据实际情况决定。一旦确定了可能的变化，就可以更有针对性地进行设计，确保未来变化发生时能够方便地扩展现有类，从而提高类的复用性。

20.2 识别变化

在软件开发过程中，程序设计必须符合用户需求。然而，在实际开发中，需求的不稳定性是一个不可避免的问题。例如，用户可能会提出新的功能需求，要求软件版本升级、平台迁移，或者适应软硬件设备的多样性等。这些需求的变化往往在开发初期难以具体化，通常会在未来的某个时间点才明确具体内容。比如，可能需要为网页添加“联系我们”的功能；或者一个购物 App 目前仅支持几种推荐算法，但未来肯定会有新的推荐算法加入；再如，当前系统仅需支持 MySQL 数据库，但将来可能需要兼容 Oracle 或 DB2 等其他数据库。

为了应对这种不确定性，程序设计应具备足够的灵活性和弹性，以便在未来需求变化时能够方便地扩展原有程序。在面向对象程序设计中，需求的变化最终会反映在类的定义和实现上。因此，设计上的灵活性主要体现在类的复用性上，即通过扩展现有类来适应变化，而不是直接修改已有的类。

要提高类的复用性，关键在于确保其定义和实现能够灵活应对未来的变化。为了适应变化，首先需要准确识别潜在的变化。以下代码展示了当需求发生变化时，现有类可能需要进行的调整。

```

1  class A {
2  public:
3      virtual ~A( ) {}
4      virtual void f(int n,int m) { /* 实现略 */ }
5      //(1). f 的参数表可能变化
6      //(2). f 的具体实现可能变化
7      //(3). 增加新的公有成员 g( );
8      //(4). 删除成员 f(int,int)
9      //(5). 更改 f(int,int)的返回类型
10     //(6). 更改 f 的访问控制为 protected/private
11 private:
12     void h( ) { /* 实现略 */ }
13     //(7). h 的参数表可能变化
14     //(8). h 的具体实现可能变化
15     //(9). 增加新的私有成员 k( );
16     //(10). 删除成员 h( )

```

```

17    //(11). 更改 h( ) 的返回类型
18    //(12). 更改 h 的访问控制为 public
19    private:
20    //(13). 增加新的数据成员
21    //(14). 数据类型可能变化
22    //(15). 数据组织可能变化
23    int  mNumbers[10];
24 };

```

以下是常见的两类可能变化：接口变化和实现变化。

接口变化指的是当变更发生时，外部类和对象与类 A 交互的方式会发生改变。由于外部只能通过公有成员与类 A 交互，因此接口变化仅限于公有部分的变化。例如，示例中的(1)、(3)、(4)、(5)、(6)、(12)属于接口变化。其中，(1)和(5)涉及接口的变更，(3)和(12)是接口的扩展，而(4)和(6)则是接口的缩窄。扩展接口不会影响现有类，但变更接口和缩窄接口则会影响现有类。对于变更接口，可以通过重新设计来避免对现有类的影响；而对于缩窄接口，它必定会影响现有类，因此在面向对象程序设计中通常不允许缩窄接口，即使编程语言的语法支持，也不应采用这种设计。

实现变化是指只影响本类中成员函数和数据的具体实现方式。由于这类变化不涉及类外部分，因此不会影响外部与类 A 的交互，也不会影响到外部现有的类。例如，示例中的(2)、(7)、(8)、(9)、(10)、(11)、(13)、(14)、(15)都属于实现变化。其中，(2)是改变公有接口的具体实现，而(7)、(8)、(9)、(10)、(11)、(13)、(14)、(15)则是对类内部私有部分的修改，即类的内部实现细节的调整。

20.3 单方向变化的适应

20.3.1 接口变化的适应

对于扩展接口的变化，可以使用继承或水平关系适应。例如，对于下面的类 A，

```

1  class A {
2  public:
3      virtual ~A( ) { }
4      virtual void f(int n,int m) { /* 实现略 */ }
5      //需要增加新的公有成员 g( );
6      /* 其它略 */
7  };

```

可以在不修改既有类 A 的前提下，通过公有继承或组合来适应变化，如

```

1  //通过公有继承定义新类 ExtA
2  class ExtA : public A {

```

```

3  public:
4      void g( ) { } // 新增的 g( )
5  };
6  //或者通过水平关系定义新类 ExtA
7  class ExtA {
8  public:
9      ExtA( ): mpA(new A) { }
10     virtual ~ExtA( ) { delete mpA;}
11     virtual void f(int n,int m) { mpA->f(n,m); }
12     void g( ) { } // 新增的 g( )
13     /* 其它略 */
14 private:
15     A * mpA;
16 };

```

对于扩展接口，两种适应方式各有优缺点。

在公有继承的方式下，优点是代码简洁，并且能够保持子类型的语义关系，即仍然可以将 ExtA 视作 A 类型的对象。然而，这种方式也存在两个主要缺点：首先，尽管 ExtA 是 A 的子类型，但由于它们的公有接口不同，类 A 的客户端可能无法直接使用 ExtA 对象，通常需要在运行时进行向下类型转换。其次，继承只能应对单一维度的变化；如果存在多个变化方向，则子类的数量会急剧增加。

采用水平关系(如依赖、组合、聚合或普通关联)时，同样存在优缺点。从代码实现的角度来看，这种方式更为复杂，例如需要处理构造函数、析构函数、复制和赋值等操作。更重要的是，新类 ExtA 将失去与原类 A 的逻辑上的子类型关系，即 ExtA 不再是 A 的一种子类型。这意味着，要使用 ExtA 对象，类 A 的客户端必须做出相应的修改，无法直接作为 ExtA 的客户端。

对于接口的变更，通常是由于在分析和设计阶段对类的行为能力考虑不周全所导致的。这种情况在设计阶段应尽量避免，但这要求设计者具备更丰富的设计经验和领域知识。如果确实需要变更接口，可以考虑通过水平关系设计一个新的类 ExtA，但不建议使用公有继承，因为这可能导致新旧类的接口发生冲突。例如：

```

1  class A {
2      virtual ~A( ) {}
3      virtual void f(int n,int m) { /* 实现略 */ }
4      //f 的参数表可能变化
5  };
6  class ExtA {
7  public:
8      ExtA( A & aA): pA(&aA) { }

```

```

9      virtual ~ExtA( ) { }
10     virtual void f( ) { /* 略 */ }
11 private:
12     A * pA;
13 };

```

实际上对于变更接口的情况,如果考虑充分,最好的方式是在设计阶段将参数的变化单独抽象出来或重构(Refactor)类 A 及与类 A 相关的类,例如:

```

1  class A {
2      virtual ~A( ) {}
3      virtual void f(int n,int m) { /* 实现略 */ }
4      //f 的参数表可能变化
5  };
6  class Client {
7  public:
8      void useA( A & a,int n,int m) { a.f(n,m); }
9  };

```

可通过将参数列表抽象为 ParameterList 类,重新设计或重构为:

```

1  class A {
2      virtual ~A( ) { }
3      virtual void f( const ParameterList & list) { /* 实现略 */ }
4  };
5  class Client {
6  public:
7      void useA( A & a, const ParameterList & list) { a.f( list ); }
8  };

```

如果需要更改函数 f 的参数列表,只需从 ParameterList 类派生新的子类即可,无需修改类 A 和 Client 类。

从上述分析可以看出,对于类的接口,保持其稳定性是最佳设计方案。若确实需要扩展接口,可以通过组合或继承来实现。而对于变更接口的需求,尽管面向切面编程(AOP)或动态代理能提供一定支持,但在面向对象程序设计中,仍应尽量避免。

20.3.2 实现变化的适应

实现上的变化是面向对象程序设计中最常见的情况。无论是公有接口实现的变化还是内部私有的变化,都不会影响外部对本类对象的访问。例如:

```

1  class A {

```

```

2  public:
3      virtual ~A( ) { }
4      virtual void f( ) { /* 具体实现可能变化 */ }
5  private:
6      Data * mpData;
7  };
8  class Client {
9  public:
10     void useA( A & a ) { a.f( ); }
11 };

```

最常用的适应方式就是从 A 类派生新的子类 ExtA，如：

```

1  class ExtA : public A {
2  public:
3      virtual void f( ) override { /* 新的实现 */ }
4  };

```

这样，当变化发生时，无需修改原类 A 或 Client 类，只需将 ExtA 的实例传递给 Client 的 useA 函数，即可执行新的实现。这里需要特别说明两点：首先，为了适应单个实现的变化，使用水平关系扩展类 A 并不是最佳选择。如果采用水平关系扩展类 A，得到的 ExtA 类将不再是 A 类的子类型，这会迫使 Client 类中的 useA 接口函数也必须进行相应的修改。其次，虽然推荐使用公有继承来扩展类 A，但这要求类 A 的函数 f() 必须是虚函数，并且类 A 的析构函数也必须是虚函数。这意味着类 A 的设计者应当预见到接口函数 f() 的实现可能会发生变化。

当然，也可以通过水平关系来适应实现的变化，例如：

```

1  class A;
2  class ImpF {
3  public:
4      virtual ~ImpF( ) {}
5      virtual void f( A & a ) = 0;
6  };
7  class UpdateImpF : public ImpF {
8  public:
9      inline virtual void f( A & a ) override;
10 };
11 class A {
12 public:
13     A( ImpF & imp ) : mpImpF( &imp ) { }
14     void f( ) { mpImpF->f( *this ); }
15 public:
16     Data * getData( ) const { return mpData; }

```

```

17 private:
18     ImpF * mpImpF;
19     Data * mpData { new Data };
20 };
21 inline void UpdateImpF::f( A & a ) {
22     a.getData( );    //模拟读写类 A 的数据
23     cout << "升级的实现\n";
24 }
25 class Client {
26 public:
27     void useA( A & aA ) { aA.f( ); }
28 };
29 int main( ) {
30     Client client;
31     UpdateImpF imp;
32     A aA( imp );
33     client.useA( aA );    //输出 升级的实现
34 }

```

在代码中，将 A 类中的 f() 函数实现抽象为 ImpF 类，并让 A 类聚合 ImpF 类。当需要新的实现时，通过从 ImpF 类派生新的子类来实现，同时保持 A 类接口的稳定性。示例中展示的只是适应实现变化的一种方案，在实际开发中，还可以根据需求选择依赖、组合或普通关联等其他形式。

20.4 分离稳定和变化

通过上一节的分析可以看出，如果类中仅存在单一方向的变化，那么通过扩展来适应这种变化相对简单。尤其是适应实现的变化通常比适应接口的变化更为容易。因此，在设计时应尽量确保公有接口的稳定性，这也是面向对象分析和设计中重点关注类的公有行为稳定性的原因。然而，在实际工程实践中，更常见的情况是类中存在多方向的变化。为了提高代码的复用性，必须对这些变化进行有针对性的设计，这使得设计过程更加复杂。

这里所说的“多方向变化”包含两层含义：一方面，接口变化和实现变化可能同时发生；另一方面，即使在接口或实现内部也可能存在多个变化方向。例如，既需要增加新的接口函数，又需要修改现有接口函数的参数列表；或者多个接口函数的实现方式同时发生变化。面对这种多方向变化，最基本的设计思想是分离稳定部分与变化部分：当类中存在多个变化方向时，可以将该类拆分为多个类，最常见的做法是创建一个稳定类和多个变化类。其中，稳定类负责表示类中的不变部分，而各个变化类则分别对应不同的变化方向，每个变化类只专注于一个方向的变化。通过这种方式，可

以构建一个局部的“网络”，其功能与原始类等价，但在类(对象)之间的关系上只有水平关系(如依赖、关联、组合、聚合)。当变化发生时，这个“网络”保持稳定，只需从网络中的某个结点派生子类即可。由于每个结点只负责一个变化方向，因此可以从多个结点派生子类以适应多个变化方向。

理想情况下，如果在分析设计阶段能够识别出所有潜在的变化，就可以构建一个稳定的“网络”来适应各种变化。然而，变化的原因多种多样，“识别全部变化”只能是理想情况。现实中，我们通常是在问题域内分析变化，并经过权衡和妥协，确定需要适应的接口变化和实现变化。在进一步的分离过程中，应尽可能保持“网络”中结点的稳定性，即确保结点的接口稳定。

接下来，我们将分别讨论针对接口变化和实现变化的分离方法。

20.4.1 分离接口变化

当存在接口变化时，要分离稳定和变化，通常两种方式：

第一种，把接口与实现分离成两个类，一个只负责接口变化，另一个只负责实现变化，这也称为桥接模式(Bridge Pattern)。这种方式中，先把接口变化看作比实现变化更“稳定”的部分，从而分成两个类，当接口变化时，从接口部分扩展，而不影响适应实现部分的变化，同样地，当实现变化时，只需扩展实现部分，不影响接口部分的变化。例如对于下面的类 Foo：

```
1  class Foo {
2  public:
3      virtual ~Foo( ) { }
4      virtual void f( ) { }
5  private:
6      void h( ) { }
7  private:
8      int mData;
9  };
```

如果类 Foo 可能有新增公有函数 g()的需求，同时修改公有 f()的实现，新增私有函数 k()，变更私有 h()的实现，修改数据成员等需求，也就是类 Foo 既有接口变化，也有实现变化的需求。采用这种方式设计的类图如图 20-1 所示。

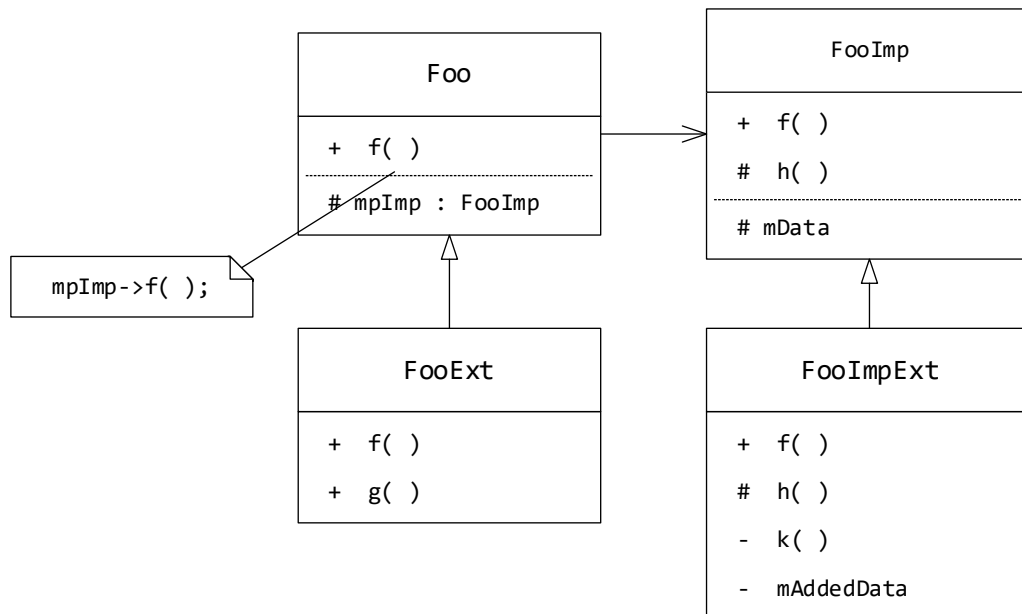


图 20-1 同时适应接口变化和实现变化的 UML 类图

具体的 C++代码实现可能如下：

```

1  class Foo {
2  public:
3      Foo(FooImp & imp ): mpImp( &imp ) { }
4      virtual ~Foo( ) { }
5      virtual void f( ) { mpImp->f( ); }
6  protected:
7      FooImp * mpImp;
8  };
9  class FooImp {
10 public:
11     virtual ~FooImp( ) { }
12     virtual void f( ) { }
13 protected:
14     virtual void h( ) { }
15 protected:
16     int mData;
17 };
18 //新增 Foo 的接口示例
19 class FooExt : public Foo {
20 public:
21     FooExt( FooImp & imp ):Foo( imp ) { }
22     virtual void g( ) { /* 新增的接口函数 */ }
23 };
24 //扩展 FooImp 的实现示例
25 class FooImpExt : public FooImp {

```

```

26 public:
27     virtual void f( ) override { /* 新的 f( )实现 */ }
28 protected:
29     virtual void h( ) override { /* 新的 h( )实现 */ }
30     void k( ) { } // 新增非公有的 k( )
31 protected:
32     int mAddedData; // 新增的非公有数据成员
33 };

```

可以看出,通过将接口和实现分离成两个类,接口部分只含有公有接口和一个指向实现的指针,实现部分就是原类的类 Foo。分离后,当接口变化时,可以从 Foo 类派生子类以适应接口变化;当实现变化时,可以从 ImpFoo 类派生子类适应实现变化。这有两点好处:首先,分离的接口和实现使得各部分可独立变化,其次,向类 Foo 的使用者隐藏了类 Foo 的具体实现细节,这一点对开发第三方类库更有价值。

第二种,针对接口变化本身进一步的分离,即根据变化的原因,对接口变化进行抽象和封装,再将接口变化从接口中单独分离出来,形成相对稳定的接口部分和变化的接口部分。

例如,下面的 Foo 类可能需要增加新的接口函数 gX() 和 hX() 等,

```

1 class Foo {
2 public:
3     virtual ~Foo( ) { }
4     virtual void func( ) { }
5     virtual void g1( ) { } //新增接口
6     virtual void g2( ) { } //新增接口
7     virtual void g3( ) { } //新增接口
8     virtual void h1( ) { } //新增接口
9     virtual void h2( ) { } //新增接口
10
11 };

```

在分离接口变化时,可以这样分离:将 g1, g2, g3, gx, 等函数封装为单独一个类 GImp,将 GImp 类的对象作为函数参数或数据成员;将 h1, h2, hx, 等函数封装为单独一个类 HImp,将 HImp 类的对象作为函数参数或数据成员。这样,新设计的类 Foo,可能如下:

```

1 class Foo {
2 public:
3     Foo( HImp & hImp ) : mpHImp( &hImp ) { }
4     virtual ~Foo( ) { }
5     virtual void func( ) { }
6     void gFunc( GImp & gImp ) { gImp.run( ); }

```

```

7     void hFunc( ) { mpHImp->run( ); }
8 private:
9     HImp * mpHImp;
10 };

```

在新的设计中，通过分离接口和稳定，将原始的 Foo 类，拆分成 Foo 类、GImp 类和 HImp 三个类，这时的 Foo 类是稳定的，当需要增加新的接口函数，如 g4() 和 h3() 时，可从 GImp 或 HImp 扩展，不影响 Foo 类的复用性。

如果类 Foo 还可能增加 kX() 的接口，甚至可以将 gX, hX 和 kX 等接口变化统一封装为类 Imp。此时的 Foo 类可能为：

```

1 class Foo {
2 public:
3     virtual ~Foo( ) { }
4     virtual void func( ) { }
5     void addFunc( Imp & imp ) { imp.run( ); }
6 };

```

第二种方式相比第一种方式，能够更好地适应接口变化，但需要更早地识别出可能的接口变化，这需要更好的领域知识和设计经验。作为第二种方式的一种特殊情况，当类中存在多个重载的函数时，也可使用此方式改进设计。例如，下面的 Foo 类有多个构造函数，且未来还会增加不同参数的构造函数：

```

1 class Foo {
2 public:
3     Foo( ) { }           //1
4     Foo( int ) { }       //2
5     Foo( A & aA ) { }     //3
6     Foo( B & aB, int ) { } //4
7 };

```

Foo 类中的接口变化是由于构造函数的参数不同造成的，为此可以将构造函数的参数列表单独封装，这时的 Foo 类可能如下：

```

1 class Foo {
2 public:
3     Foo( ) { }           //1
4     Foo( int ) { }       //2
5     Foo( ComplexParameters & params ) { } //5
6 };

```

ComplexParameters 类是对复杂参数列表的抽象，针对不同参数列表可从 ComplexParameters 类派生不同子类，这样类 Foo 的构造函数就是稳定的。例子中，只是对 3 和 4 中的参数列表进行了抽象和封装，因此当前的 ComplexParameters 类可以只有两个子类，分别对应//3 和//4 中的参数列表，当增加新参数的构造函数时，只需从 ComplexParameters 类派生新子类，不影响类 Foo 的复用性。当然，如果有必要，也可以对所有构造函数的参数列表进行抽象和封装。

20.4.2 分离实现变化

当只有实现变化时，无论是私有实现细节的变化还是多个接口实现的变化，仍可以采用类似分离接口变化的方式，分离实现变化的稳定部分和变化部分。

第一种，由于只有实现变化，那么相对地接口就是稳定的，可以将接口作为稳定的类，而将实现变化抽象到另一个类，以此来实现分离。例如，对于下面的类 Foo：

```

1  class Foo {
2  public:
3      void f( ) { /* 实现可能变化 */ }
4      void g( ) { /* 实现可能变化 */ }
5  private:
6      void k( ) { /* 可能变化或新增私有成员 */ }
7  private:
8      int  mData; /* 私有实现细节可能变化 */
9  };

```

可以这样改进设计：

```

1  class Foo {
2  public:
3      Foo( FooImp & imp ) : mpImp( &imp ) { }
4      void f( ) { mpImp->f( ); }
5      void g( ) { mpImp->g( ); }
6  private:
7      FooImp * mpImp;
8  };
9  class FooImp {
10 public:
11     virtual ~FooImp( ) { }
12     virtual void f( ) { /* 略 */ }
13     virtual void g( ) { /* 略 */ }
14 protected:
15     virtual void k( ) { /* 略 */ }
16 protected:

```

```

17     int mData;
18 };

```

这里，将 Foo 类中的所有实现变化封装到 FooImp 类，Foo 类只保留公有接口以及指向实现的指针，当实现发生变化时，只需扩展 FooImp 类，不需要修改 Foo 类，同时对 Foo 类的使用者隐藏了具体实现细节。这种设计方式也称 PImpl (Pointer to Implementation)，实际上是前面提到的桥接模式的一个特例，即通过一个指向具体实现的指针，分离了稳定的接口部分和变化的实现部分，使得适应实现变化不影响接口的稳定，也就保证了类 Foo 的复用性，这种分离接口和变化的方式在设计中也是常见的，例如，系统库中的 string 类的设计就是这样的：

```

1  class string {
2  public:
3      Xxx func1( ) { return mpImp->func1( ); }
4      Yyy func2( ) { return mpImp->func2( ); }
5  private:
6      StringImp * mpImp;
7  };

```

但这种方式并没有从本质上解决分离实现变化的问题，当存在多方向的实现变化时，FooImp 类如何适应还需要进一步讨论，这就是第二种方式。

第二种，当类中存在多个方向的实现变化时，直接通过虚机制派生子类是不合适的。如前章所述，直接使用公有继承和虚机制可能导致子类数量呈几何级数增长。为此，作为一般方法，可以先将变化分为接口实现的变化和非公有的内部实现细节的变化，再针对这两种变化类型分别进行设计，以分离稳定部分和变化部分。对于接口实现的变化，可将每个接口的实现变化独立封装；而对于非公有的内部实现变化，则可以统一封装到一个单独的变化类中。例如，对于下面的类 Foo：

```

1  class Foo {
2  public:
3      virtual ~Foo( ) { }
4      virtual void f( ) { /* 变化方向 1:f( )的实现可能变化 */ }
5      virtual void g( ) { /* 变化方向 2:g( )的实现可能变化 */ }
6      virtual void h( ) { /* 变化方向 3:h( )的实现可能变化 */ }
7  private:
8      void k( )    { /* 变化方向 4:k( )的实现可能变化 */ }
9      //void w( ) { /* 变化方向 5:增加新的私有成员函数 w( ) */ }
10 private:
11     int  mData; /* 变化方向 6:添加、修改私有的数据成员 */
12 };

```

类 Foo 中存在多个方向的实现变化，将各变化方向独立封装后的设计可能为：

```

1  //重新设计后的 Foo 类
2  class Foo {
3  public:
4      void f( ) { mpImpF->f( *mpPrivate ); }
5      void g( ) { mpImpG->g( *mpPrivate ); }
6      void h( ) { mpImpH->h( *mpPrivate ); }
7  private:
8      ImpF * mpImpF;
9      ImpG * mpImpG;
10     ImpH * mpImpH;
11     ImpPrivate * mpPrivate;
12 };
13 //所有的私有变化封装到 ImpPrivate 类
14 class ImpPrivate {
15 public:
16     virtual ~ImpPrivate( ) { }
17     virtual void k( ) { }
18     int mData;
19 };
20 //接口函数 f( ) 的所有实现变化封装到 ImpF 类
21 class ImpF {
22 public:
23     virtual ~ImpF( ) { }
24     virtual void f( ImpPrivate & data ) { /* f( ) 的实现 */ }
25 };
26 //接口函数 g( ) 的所有实现变化封装到 ImpG 类
27 class ImpG {
28 public:
29     virtual ~ImpG( ) { }
30     virtual void g( ImpPrivate & data ) { /* g( ) 的实现 */ }
31 };
32 //接口函数 h( ) 的所有实现变化封装到 ImpH 类
33 class ImpH {
34 public:
35     virtual ~ImpH( ) { }
36     virtual void h( ImpPrivate & data ) { /* h( ) 的实现 */ }
37 };

```

从代码中可以看出，当需要修改函数 f() 的实现时，可以通过从 ImpF 类公有派生出新的子类，并在其中提供 f() 的新实现；对于 g() 和 h() 的变化，也可以采用相同的方法。当私有部分需要调整时，由于这些私有成员对外不可见，因此不会影响 Foo 类的使用者，处理这部分变化更

为灵活。这里，所有私有内容被封装到 `ImpPrivate` 类中。总之，通过分离稳定的接口和多个可变的类，在实现发生变化时，只需扩展 `ImpF`、`ImpG`、`ImpH` 和 `ImpPrivate` 类即可适应这些变化，同时保持 `Foo` 类的复用性不受影响。

表面上看，这种分离实现变化的方法似乎简单直接，但这只是最基础的通用策略，并且依赖于一个隐含的前提：即各个变化方向是相互独立的。然而在实际开发中，这种独立性往往难以保证，各变化方向之间可能存在复杂的相互影响。要实现真正的独立变化，通常需要更深入的分析 and 精心设计，包括如何合理抽象行为、详细分析变化的具体原因以及评估不同实现之间的差异等。

例如下面的类 `Foo`：

```
1  class Foo {
2  public:
3      virtual ~Foo( ) { }
4      virtual void func1( ) {
5          doInit1( );
6          do1( );
7          doTerm1( );
8      }
9      virtual void func2( ) {
10         doInit2( );
11         do2( );
12         doTerm2( );
13     }
14 };
```

`func1` 和 `func2` 都是虚函数，这表明在 `Foo` 的子类中可能需要重写它们的实现。在具体设计时，有多种方案可供选择。例如，可以将 `func1` 和 `func2` 的实现单独抽象出来，代码可能会这样：

```
1  class Foo {
2  public:
3      virtual ~Foo( ) { }
4      virtual void func1( Func1Imp & imp1 ) { imp1.run( ); }
5      virtual void func2( ) { mpImp2->run( ); }
6  private:
7      Func2Imp * mpImp2;
8  };
```

还可以将 `func1` 和 `func2` 的实现抽象成一个类，代码可能会这样：

```
1  class Foo {
2  public:
3      virtual ~Foo( ) { }
```

```

4     virtual void func( FuncImp & imp ) { imp.run( ); }
5 };

```

如果 doInit1() 与 doInit2() 相同, doTerm1() 与 doTerm2() 也相同, 唯一的区别在于 do1() 和 do2(), 那么可以将 do1 和 do2 的部分单独抽象出来。代码可能会这样:

```

1  class Foo {
2  public:
3      virtual ~Foo( ) { }
4      virtual void func1( Do1Imp & imp1 ) {
5          doInit( );
6          imp1.run( );
7          doTerm( );
8      }
9      virtual void func2( ) {
10         doInit( );
11         mpImp2->run( );
12         doTerm( );
13     }
14 private:
15     Do2Imp * mpImp2;
16 };

```

甚至可以将 do1 和 do2 部分抽象成一个 DoImp 类, 这时代码可能这样:

```

1  class Foo {
2  public:
3      virtual ~Foo( ) { }
4      virtual void func1( DoImp & imp ) {
5          doInit( );
6          imp.run( );
7          doTerm( );
8      }
9      virtual void func2( ) {
10         doInit( );
11         mpImp->run( );
12         doTerm( );
13     }
14 private:
15     DoImp * mpImp;
16 };

```

更进一步, 还可以合并 func1 和 func2 两个函数为一个函数 func, 代码可能这样:

```

1  class Foo {

```



```

2 public:
3     virtual ~Foo( ) { }
4     virtual void func( DoImp & imp ) {
5         doInit( );
6         imp.run( );
7         doTerm( );
8     }
9 };

```

20.5 分离及适应变化

分离变化并没有一种固定的设计方法，因为变化的原因和形式多种多样，因此可以采用不同的设计方案。这些方案之间的差异往往体现在合适的程度上，而判断其合适性的标准在于能否更好地适应未来的变化。然而，无论具体如何变化，设计的核心思想始终不变：通过水平关系将稳定部分与变化部分进行分离。分离之后，便可以利用面向对象的动态多态性，借助公有继承来应对实现的变化。在 C++ 中，这可以通过运用虚函数机制来实现。接下来，我们将通过一些具体的代码片段，演示在 C++ 中如何实现这种分离与适应变化的方法。

20.5.1 使用依赖和继承

依赖关系是一种实例间水平关系，A 依赖 B，通常表示 A 中的某个功能的实现依赖 B 类的实例对象。在依赖关系中，变化可以来自 B，也可以来自 A，也就是 B 可以是多变的，也可能 A 是多变的。对于多变的一方，可以通过继承以子类型化的方式扩展既有实现，从而既能适应变化，有能保证其它类的稳定。在使用依赖和继承时，类之间的基本关系形式如图 20-2 所示。

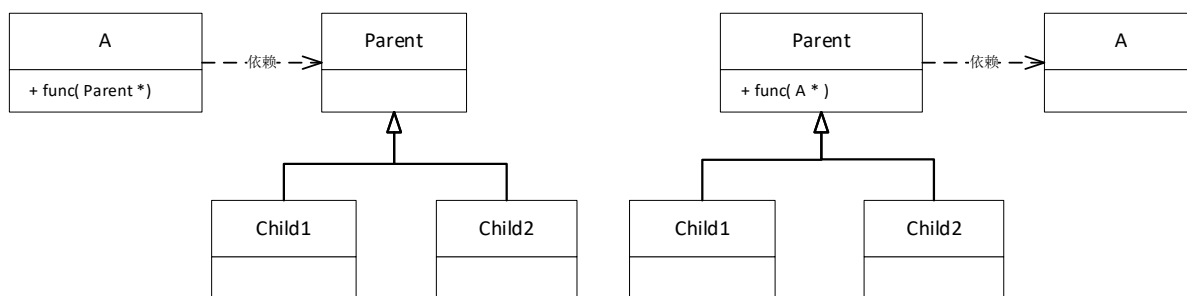


图 20-2 使用依赖和继承的基本形式

此时 Parent 类是易变的，为适应变化可从 Parent 类派生新的子类，如 Child3，这样 Parent 的各类子类针对不同的需求会有不同的实现，并以此来适应需求的变化。而在适应的过程中，A 类、Parent 类以及既有的子类 Child1 和 Child2 的定义和实现都不需要修改，从而保证既有类的复用性。

在实际项目中，使用依赖和继承可能有多种变化形式。下面针对一些常见形式，通过示例给出

简要说明。

例 1: 在依赖关系中，多变的可能是依赖方，也可能是被依赖方。被依赖方作为多变部分是最为常见的形式。例如，在“小猫钓鱼”游戏中，一只小猫钓到不同的鱼，会有不同的经验奖励，那么鱼就是多变的。此时类之间的关系可通过如图 20-3 所示的 UML 类图进行表示：

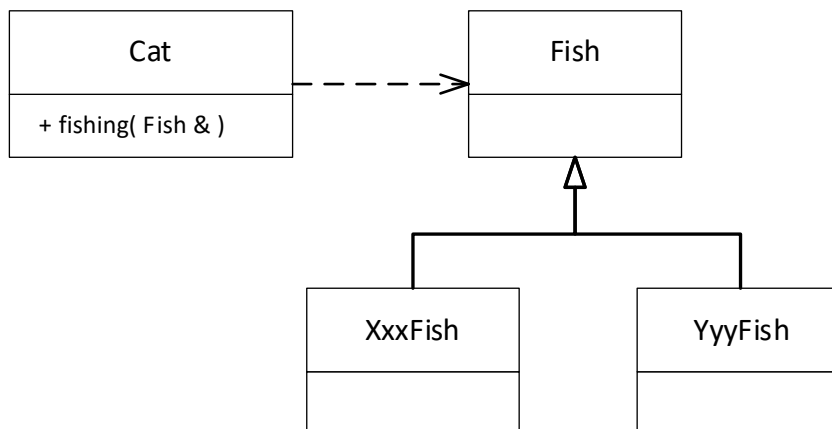


图 20-3 依赖和继承中鱼多变时的 UML 类图

示意代码如下：

```

1  class Fish {
2  public:
3      virtual ~Fish( ) = default;
4      virtual int getExp( ) const = 0;
5  };
6  class XxxFish :public Fish {
7  public:
8      virtual int getExp( ) const override { return 100; }
9  };
10 class YyyFish :public Fish {
11 public:
12     YyyFish( int w ) :mWeight( w ) {}
13     virtual int getExp( ) const override { return 50 + mWeight; }
14 private:
15     int mWeight;
16 };
17 class Cat {
18 public:
19     void fishing( const Fish& fish );
20 private:
21     int mLevel = 0;
22     int mExp = 0;
23 };
  
```

```

24 inline void Cat::fishing( const Fish& fish ) {
25     mExp += fish.getExp( );
26     mLevel = mExp / 100;
27 }

```

例 2：在依赖关系中，依赖方作为多变部分也是可以的。例如，在“小猫钓鱼”游戏中，如果猫有多种，而鱼只有一种，不同类型的猫钓到鱼，获得的奖励效果不同。在这里，猫是多变的，不同种类的猫有着各自独特的钓鱼方式，也就是说，每种猫的钓鱼行为各不相同。当需要添加新种类的猫时，只需从 Cat 类派生一个新的子类，并在新子类中重写（override）父类中的 fishing(const Fish &)方法即可。在此过程中无需对 Cat 类以及已有的 Fish 类做任何修改，从而保证了 Cat 类和 Fish 类的复用性。此时类之间的关系可通过如图 20-4 所示的 UML 类图进行表示：

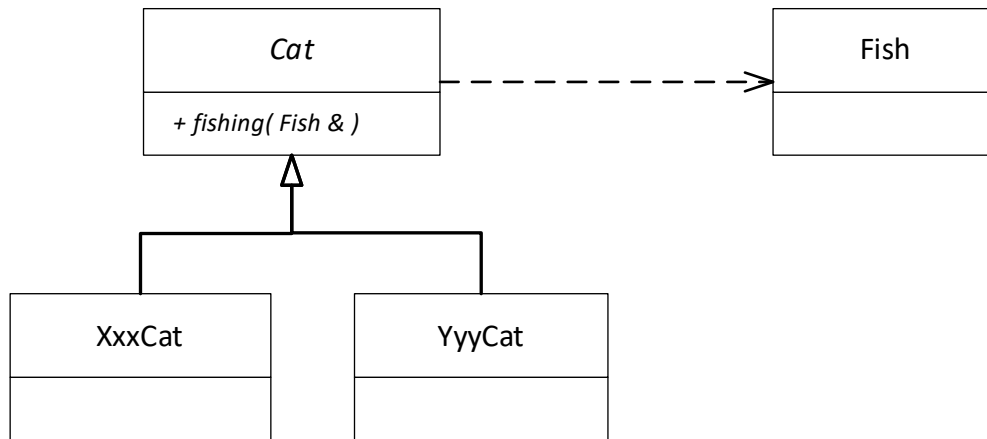


图 20-4 依赖和继承中猫多变时的 UML 类图

示意代码如下：

```

1  class Fish {
2  };
3  class Cat {
4  public:
5      virtual ~Cat( ) = default;
6      virtual void fishing( const Fish& fish ) = 0;
7  protected:
8      int mLevel = 0;
9      int mExp = 0; //本级已积攒的经验
10 };
11 class XxxCat :public Cat {
12 public:
13     virtual void fishing( const Fish& fish ) override;
14 };

```

```

15 class YyyCat :public Cat {
16 public:
17     virtual void fishing( const Fish& fish ) override;
18 };
19 //XxxCat 每钓一条鱼，就升一级
20 inline void XxxCat::fishing( const Fish& fish ) {
21     ++mLevel;
22 }
23 //XxxCat 每钓一条鱼，增加 80 点经验，每 100 点经验升一级
24 inline void YyyCat::fishing( const Fish& fish ){
25     mExp += 80;
26     mLevel = mExp / 100;
27     mExp = mExp % 100;
28 }

```

例 3：依赖方和被依赖方也可以同时变化。例如，每种猫钓到不同的鱼，采用同样的算法进行升级。这里，猫和鱼都是多变的，此时类之间的关系可通过如图 20-5 所示的 UML 类图进行表示：

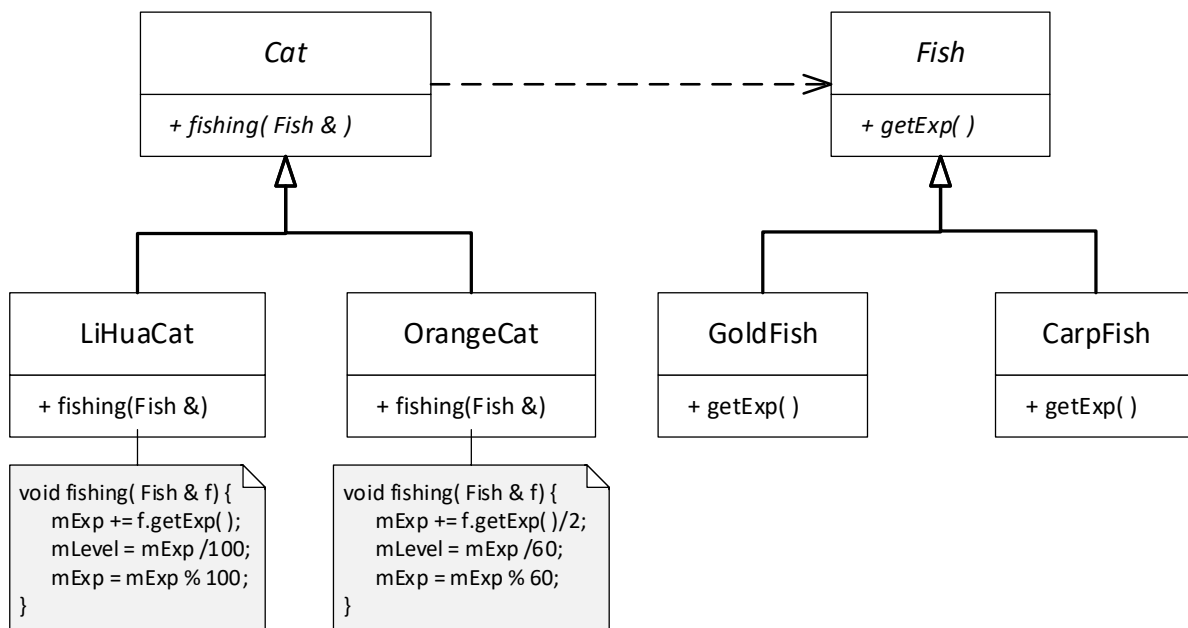


图 20-5 依赖和继承中猫和鱼同时多变的 UML 类图

示意代码如下：

```

1 class Fish {
2 public:
3     virtual ~Fish( ) = default;
4     virtual int getExp( ) const = 0;
5 };

```

```

6  class GoldFish :public Fish {
7  public:
8      virtual int getExp( ) const override { return 100; }
9  };
10 class CarpFish :public Fish {
11 public:
12     CarpFish( int w ) :mWeight( w ) {}
13     virtual int getExp( ) const override { return 50 + mWeight; }
14 private:
15     int mWeight;
16 };
17 class Cat {
18 public:
19     virtual ~Cat( ) = default;
20     virtual void fishing( const Fish& fish ) = 0;
21 protected:
22     int mLevel = 0;
23     int mExp = 0;
24 };
25 class LiHuaCat :public Cat {
26 public:
27     virtual void fishing( const Fish& fish ) override;
28 };
29 class OrangeCat :public Cat {
30 public:
31     virtual void fishing( const Fish& fish ) override;
32 };
33 inline void LiHuaCat::fishing( const Fish& fish ) {
34     mExp += fish.getExp( );
35     mLevel = mExp / 100;
36     mExp = mExp % 100;
37 }
38 inline void OrangeCat::fishing( const Fish& fish ) {
39     mExp += fish.getExp( )/2;
40     mLevel = mExp / 60;
41     mExp = mExp % 60;
42 }

```

上述代码中，对于每种猫而言，无论钓到何种鱼，其升级算法均保持一致，因此，可以随时扩展新的猫和鱼的子类，以应对未来的新功能需求。然而，新增子类属于未来可能发生的变化，而届时是否仍能保持算法的一致性则具有不确定性。所以，这种设计存在一定的局限性，具体分析详见例 4。

例 4: 同时变化的局限性。假设猫和鱼都是多变的, 那么受影响的是 `fishing` 操作的行为效果。如果 `fishing` 的算法是统一的, 例如无论哪种猫钓到哪种鱼都统一升级 1 级, 那么应对这种变化是相对简单的。如果 `fishing` 的算法仅依赖于猫或鱼其中一方的类型, 那么通过继承与动态多态即可实现, 例 3 正是属于这种情形。但如果 `fishing` 的算法与猫和鱼两方类型都相关, 那么要适应变化就变得困难了。

以例 3 为基础继续说明, 假设现在新增一种类型的鱼—草鱼 (`GrassFish`), 并且规定: 当狸花猫钓到草鱼时, 直接升 2 级; 而橘猫钓到草鱼时, 则升 3 级。在这种情况下, 每种猫钓到不同种类鱼的升级规则既受猫自身类型的影响, 也受鱼类型的影响。为了适应这种双重影响带来的变化, 可以采用以下两种方式:

第一种, 在 `fishing` 函数中使用条件分支语句, 根据鱼的具体动态类型来实现相应的升级逻辑, 如:

```

1 void LiHuaCat::fishing( const Fish& fish ) {
2     if( const GrassFish * p = dynamic_cast<const GrassFish*>( &fish ) ) {
3         mLevel += 2;
4     } else {
5         mExp += fish.getExp( );
6         mLevel = mExp / 100;
7         mExp = mExp % 100;
8     }
9 }
10 void OrangeCat::fishing( const Fish& fish ) {
11     if( const GrassFish * p = dynamic_cast<const GrassFish*>( &fish ) ) {
12         mLevel += 3;
13     } else {
14         mExp += fish.getExp( )/2;
15         mLevel = mExp / 60;
16         mExp = mExp % 60;
17     }
18 }

```

但这种对 `LiHuaCat` 类和 `OrangeCat` 类的代码修改, 会破坏既有类的复用性。

第二种, 仍然使用继承和动态多态, 但需要为各种猫类分别派生出专门针对草鱼的子类。此时的 UML 类图如图 20-6 所示:

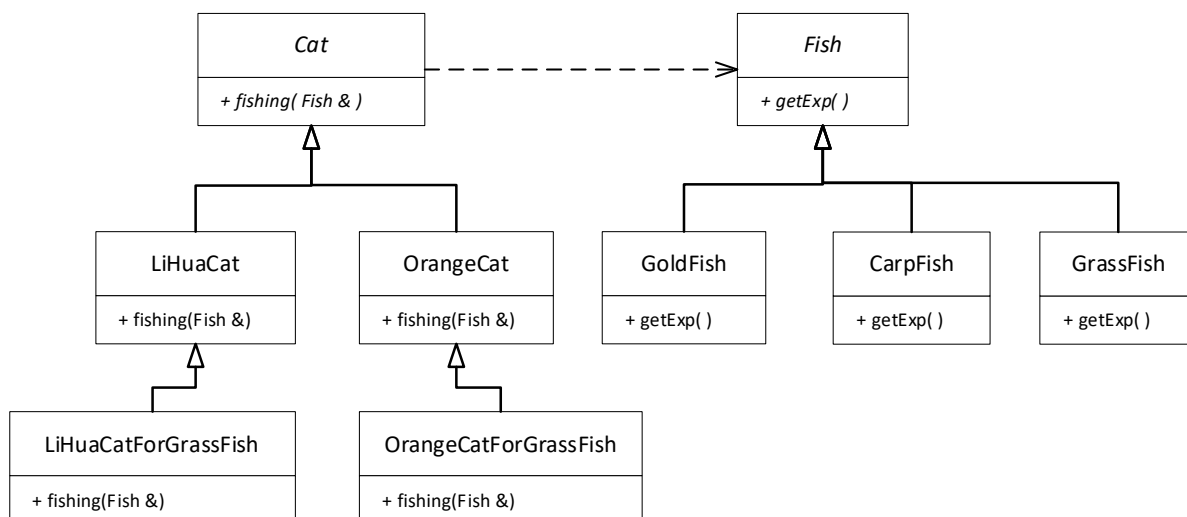


图 20-6 依赖和继承中新增子类草鱼的 UML 类图

示意代码如下：

```

1  class LiHuaCat : public Cat {
2  public:
3      virtual void fishing( const Fish& fish ) override;
4  };
5  class OrangeCat : public Cat {
6  public:
7      virtual void fishing( const Fish& fish ) override;
8  };
9  //新增子类 LiHuaCatForGrassFish
10 class LiHuaCatForGrassFish : public LiHuaCat {
11 public:
12     virtual void fishing( const Fish& fish ) override { mLevel += 2; }
13 };
14 //新增子类 OrangeCatForGrassFish
15 class OrangeCatForGrassFish : public OrangeCat {
16 public:
17     virtual void fishing( const Fish& fish ) override { mLevel += 3; }
18 };
  
```

假设存在 M 种猫和 N 种鱼，在这种方式下，钓鱼行为就有 $M \times N$ 种不同的实现方式。每当新增一种猫时，可能就需要额外实现 N 种钓鱼行为，通过子类实现，则意味着要每种猫要增加 N 个针对不同鱼的子类；同样地，每增加一种鱼，又可能需要为每种猫新增 M 个子类。这种以几何级数增长的子类数量，在实际开发中几乎是不可行的。

这种现象是否表明动态多态性存在缺陷呢？实际上，动态多态性有两类表现形式：一是根据目标对象的运行时类型以及方法参数列表中参数的声明类型来确定具体执行的代码段；二是依据目标

对象与参数的实际运行时类型来决定执行哪段代码。当前面向对象语言仅支持第一种形式的动态多态性，而不支持第二种。上述问题的根源在于语言实现层面对于动态多态性的支持不足，而非动态多态性本身存在问题。

在实际应用中，如果猫和鱼的行为确实多变，应该如何设计呢？首先，应考虑分析和抽象的合理性。其次，如果确实需要应对这种变化，那么需求方和开发团队可以各做一定的妥协：需求方应尽量限定猫和鱼的种类范围，确保至少有一方是相对稳定的。例如，鱼的种类可以多种，但其种类数量保持稳定，而猫的种类不受限制，与此同时，开发团队可以通过语言中的静态多态性和动态多态性来适应这些新需求，这正是设计模式中访问者模式的一个典型应用场景，此处不再深入探讨。

例 5：在依赖关系中，还可以依赖自身类对象，甚至子类对象依赖父类对象。例如：有多种怪物，怪物之间会发生一对一的战斗，不同种类的怪物有自己特有的攻击方式。此时的 UML 类图如图 20-7 所示：

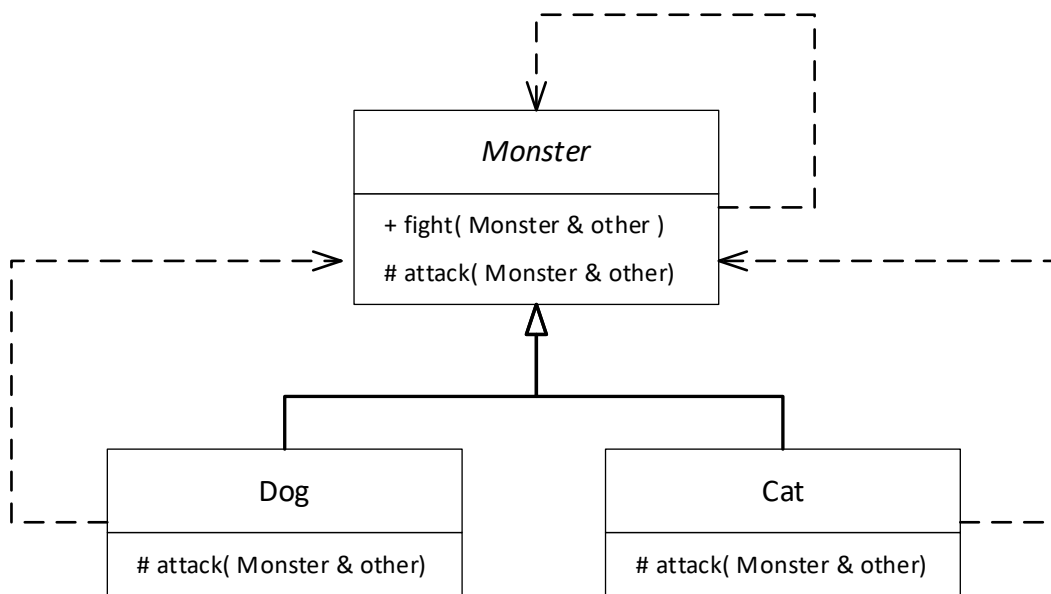


图 20-7 依赖和继承中自依赖及子类依赖父类的 UML 类图

示意代码如下：

```

1  class Monster {
2  public:
3      virtual ~Monster() = default;
4      bool fight( Monster & other); //Monster 依赖自身类
5  protected:
6      virtual bool attack(Monster & other) = 0;
7  };
8  //怪物之间的战斗，Monster 依赖 Monster
  
```



```

9  inline bool Monster::fight( Monster & other) {
10     while(true) {
11         if( attack(other) ) return true;
12         if( other.attack(*this) ) return false;
13     }
14 }
15 //Dog 作为 Monster 的子类型，也依赖 Monster 父类
16 class Dog: public Monster {
17 protected:
18     virtual bool attack(Monster & other) override {
19         // ...
20     }
21 };
22 //Cat 作为 Monster 的子类型，也依赖 Monster 父类
23 class Cat: public Monster {
24 protected:
25     virtual bool attack(Monster & other) override {
26         // ...
27     }
28 };

```

20.5.2 使用关联和继承

在关联关系中，无论是普通关联、组合还是聚合，均体现了类之间的水平关系。这些关系在代码形式上较为相似，差异主要体现在逻辑层面。当类 A 与类 B 建立关联时，变化可能来源于 A，也可能来源于 B。如果 A 和 B 同时发生变化，通常可以通过对其中一个类进行进一步拆分，将整体结构分解为更多职责更单一的类，从而在两者的关系中实现一方稳定、一方可变的设计。在使用关联（包括普通关联、组合与聚合）和继承时，类之间的基本关系形式如图 20-8 所示。

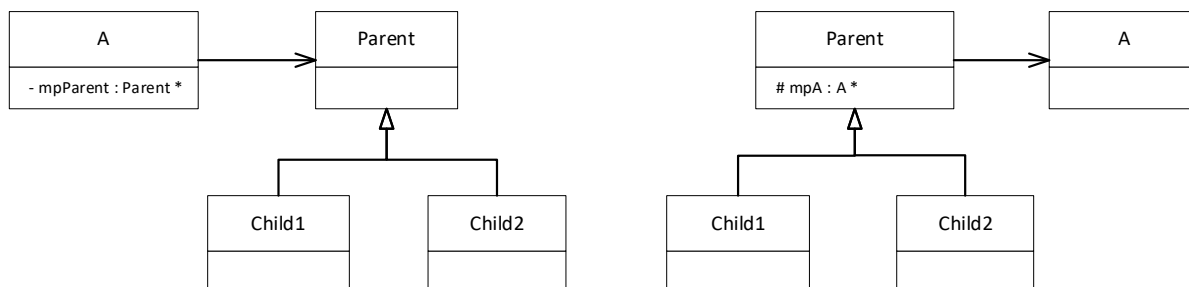


图 20-8 使用关联和继承的基本形式

在实际项目中，使用关联和继承可能有多种变化形式。下面针对一些常见形式，通过示例给出简要说明。

例 1：警察(Police)使用交通设备(Vieche)来追踪匪徒，其中交通设备会有多种，未来一定会增加新的交通设备。这里的变化体现在交通设备上，而警察类是相对稳定的。类之间的关系可通过如图 20-9 所示的 UML 类图进行表示：

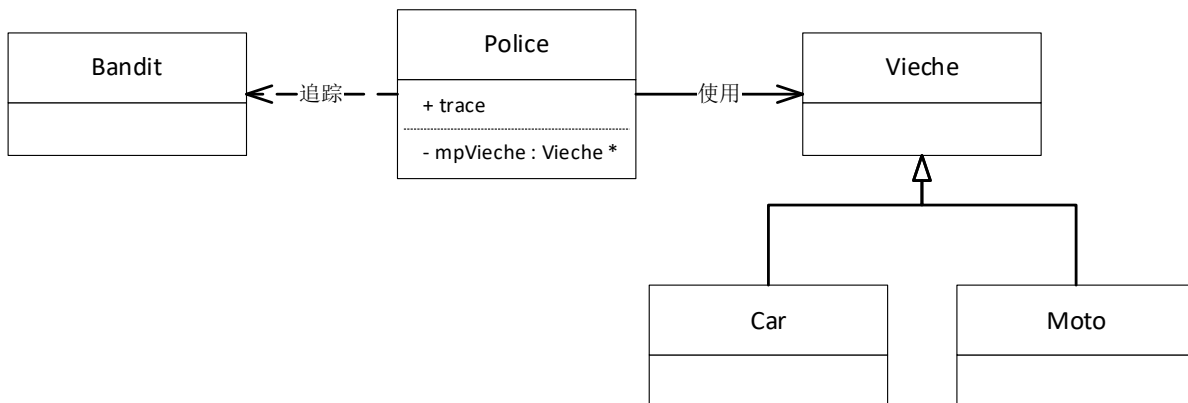


图 20-9 关联和继承中交通设备多变的 UML 类图

示意代码如下：

```

1  class Police {
2  public:
3      bool trace( Bandit & bandit, int distance, int hours ) {
4          int reduce = (mpVieche->maxSpeed( ) - bandit.speed( ))* hours;
5          return (reduce >= distance) ;
6      }
7  private:
8      Vieche * mpVieche;
9  };
  
```

例 2：科学家需要借助计算机完成面积的计算，计算机有多种，且未来还会增加新种类的计算机。这里，计算机是易变的，科学家的计算功能可通过委托完全交给 Computer 完成，也就是将函数功能委托给另一个类的对象实例(如 Computer 类及子类对象)，以此来保证科学家类的稳定。类之间的关系可通过如图 20-10 所示的 UML 类图进行表示：

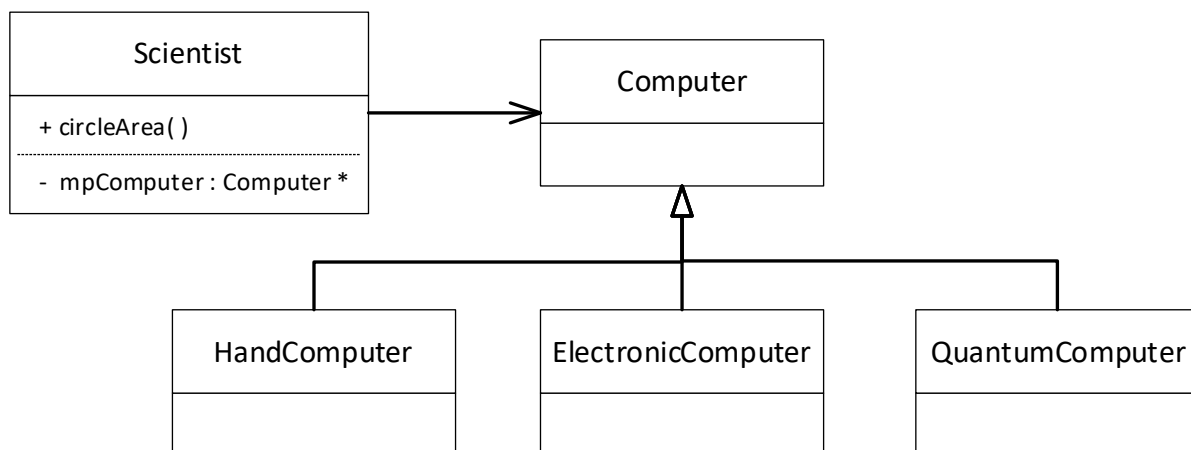


图 20-10 关联和继承中委托给不同计算机的 UML 类图

示意代码如下：

```

1  class Scientist {
2  public:
3      void setComputer( Computer & comp ){ mpComputer = &comp; }
4      float circleArea( float r ) {
5          return mpComputer->caculateArea( r );
6      }
7  private:
8      Computer * mpComputer;
9  };
  
```

例 3：使用继承能够应对变化，但引发变化的原因往往多种多样。例如，示例 2 中是为了适应计算工具的变化，下面我们再举一个为适应参数变化而设计的例子。用户（User）类需要实现登录（login）功能，登录时涉及多个参数，如用户名、密码、验证码等。考虑到未来可能出现的新需求，这些参数项可能需要扩展，例如增加 MAC 地址、IP 地址、令牌、生物识别信息等。为了保持 User 类的稳定性，可以将参数集合抽象为一个或多个独立的登录信息类，User 类通过依赖或关联这些信息类来实现功能解耦。当需要新增登录参数时，只需扩展对应的信息类，而无需修改 User 类的实现。若采用关联关系，类之间的结构可以如图 20-11 所示。

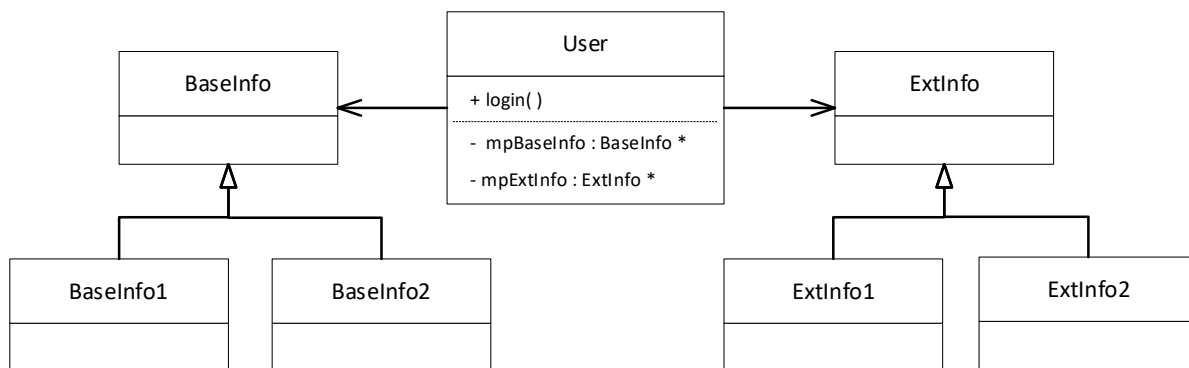


图 20-11 关联和继承中参数多变的 UML 类图

示意代码如下：

```

1  class User {
2  public:
3      bool login( ) {
4          //通过 mpBaseInfo 和 mpExtInfo 获取登录信息
5          //...
6      }
7      void setBaseInfo( BaseInfo & info )    { mpBaseInfo = &info; }
8      void setExtInfo( ExtInfo & info ) { mpExtInfo = &info; }
9  private:
10     BaseInfo* mpBaseInfo;
11     ExtInfo* mpExtInfo;
12 };
  
```

例 4：篮子中可以放置多种水果，而水果的种类可能在未来进一步扩展。在设计时，可以让 Basket 组合 Fruit，为了提高 Basket 的复用性，可以将 Basket 中创建各类水果的行为封装到一个独立的 FruitCreator 类中。这样一来，当需要新增水果种类时，只需修改或扩展枚举类 FruitType 和 FruitCreator 类，而现有的 Basket 类、Fruit 类及其子类均无需改动。类之间的关系可如图 20-12 所示：

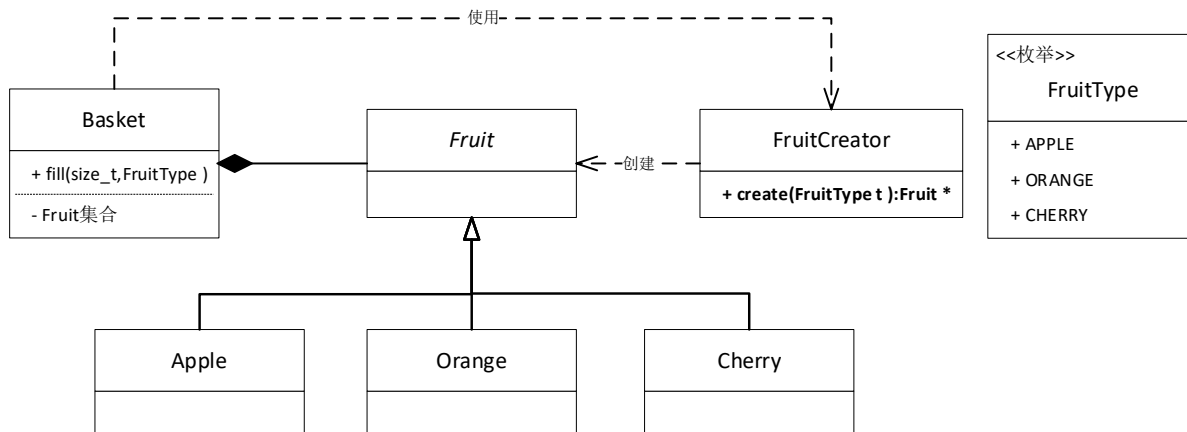


图 20-12 关联继承中水果多多变的 UML 类图

示意代码如下：

```

1  class Basket {
2  public:
3      Basket( ) { }
4      void fill(size_t index, FruitType type) {
5          mpFruits[ index ] = FruitCreator::create( type );
6      }
7      ~Basket( ) {
8          for (int i = 0; i < 5; i++)
9              delete mpFruits[ i ];
10     }
11 private:
12     Fruit * mpFruits[ 5 ] {}
13 };

```

例 5：在通过关联和继承分离稳定与变化部分时，可以利用继承来应对变化的部分。然而，相对稳定的类也可以作为其他变化类的子类。例如，某个应用程序包含多种子类的变化形式，其中子类 **MyApp** 的具体实现可能有多种不同版本，而这些不同版本又通过 **XApp** 的不同子类来实现。类之间的关系可如图 20-13 所示：

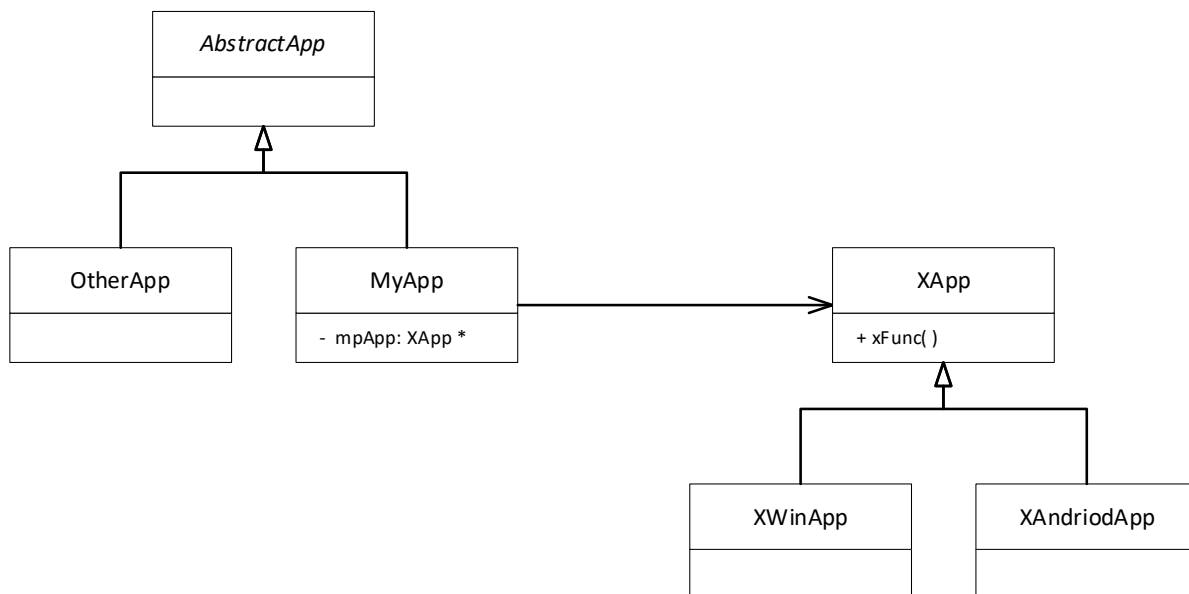


图 20-13 关联和继承中子类关联变化部分的 UML 类图

示意代码如下：

```

1  class MyApp : public AbstractApp {
2  public:
3      MyApp( XApp & xapp ) :mpApp( &xapp ) { }
4      void work( ) {
5          doSomething( );
6          mpApp->xFunc( );
7      }
8      void doSomething( ) { }
9  private:
10     XApp * mpApp;
11 };
  
```

例 6：关联还可以是自关联。如学生可以有多个研究生同学，研究同学可分硕士同学和博士同学，也可能有其它的类型同学，甚至博同学也可能有进一步的细分，如工学博士、理学博士等。通过学生类聚合学生类，可以保证学生类的稳定，又不影响学生子类型的扩展。类之间的关系可如图 20-14 所示：

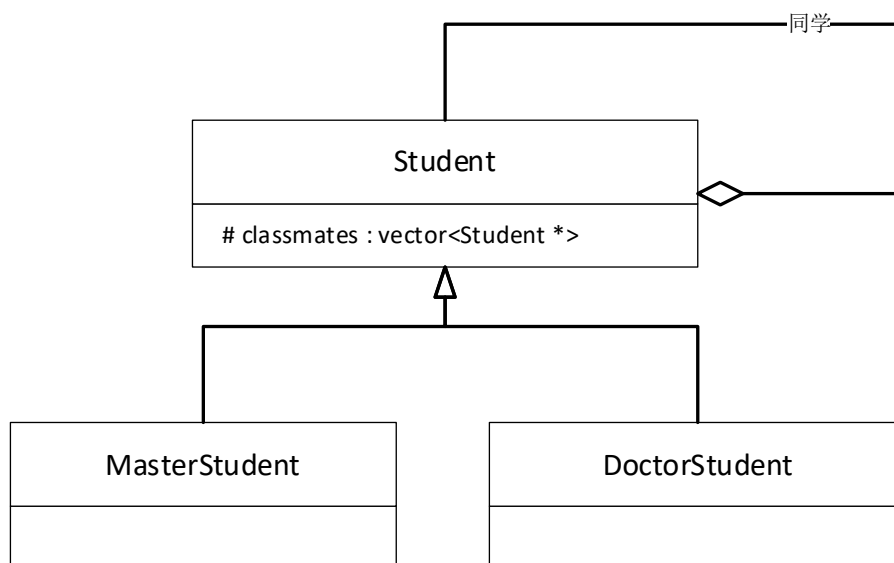


图 20-14 关联和继承中自关联且多变的 UML 类图

示意代码如下：

```

1  class Student {
2  public:
3      void add( Student & stu ) { mClassmates.push_back( &stu ); }
4  protected:
5      std::vector<Student *> mClassmates;
6  };
  
```

例 7：子类还可以与父类建立关联关系。例如，一家公司可能包含多种类型的部门，如财务部、人力资源部等，而某个部门本身也可以是一个子公司或分公司。进一步地，这些子公司或分公司又可以拥有自己的部门、子公司或分公司。由此可见，公司的组织结构本质上是一种树状结构。为了准确描述这种结构，并同时支持其灵活调整，可以定义一个抽象的“部门”类，并从该抽象类派生出具体的部门类以及部门集合类。部门集合类可以通过组合或聚合的方式包含其他部门。这样一来，实例化总公司、子公司或分公司的对象，实际上就是实例化部门集合类。类之间的关系可如图 20-15 所示：

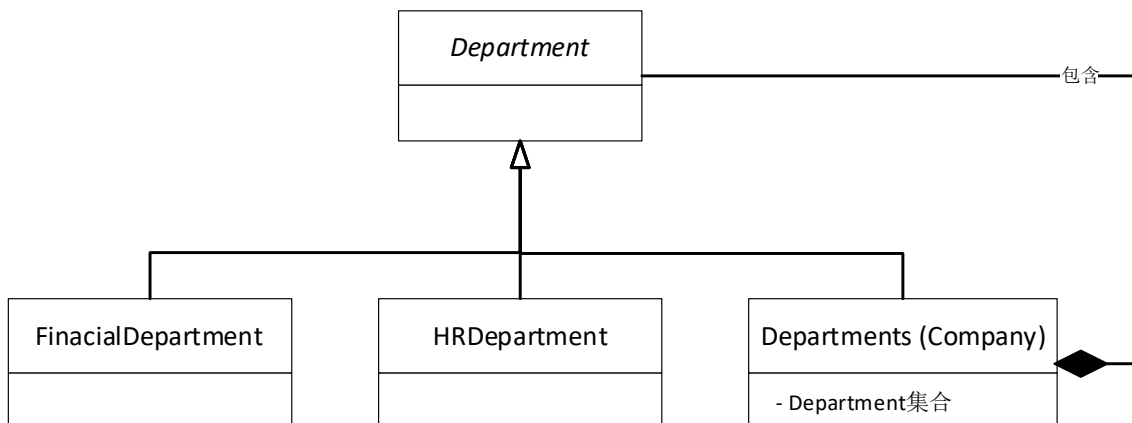


图 20-15 关联和继承中子类组合父类的 UML 类图

示意代码如下：

```

1  class Department {
2  public:
3      virtual ~Department( ) = default;
4  };
5  class HRDepartment :public Department {
6  };
7  class Departments : public Department {
8  public:
9      virtual ~Departments( ) {
10         for (auto dpt : mDepartments)
11             delete dpt;
12     }
13     void addDepartment( Department & dpt ) {
14         mDepartments.push_back( dpt.clone( ) );
15     }
16 private:
17     std::vector<Department*> mDepartments;
18 };

```

20.6 设计案例

20.6.1 回调

回调是一种编程机制，当客户端对象 A 向服务端对象 B 发送请求时，通常由服务端处理后，返回结果给客户端。服务端在处理请求的过程中，可能还需要调用一些子步骤或子过程，如果服务端本身“知道”这些子步骤或子过程，那么就是常规的过程调用了；如果这些子步骤或子过程是由客

户端指定的，那么就是回调了。

以 Windows API 中的 EnumFontsA 函数为例，其函数原型为：

```
int EnumFontsA(HDC hdc, LPCSTR lpLogfont, FONTENUMPROCA lpProc, LPARAM lParam);
```

该函数用于枚举指定设备上的可用字体。第三个参数 lpProc 允许客户端提供一个自定义的处理函数，用于决定如何处理每种字体。每当枚举到一种字体时，服务端会调用这个客户端提供的函数来执行特定操作，如预览字体。这种方式使得在不修改 EnumFontsA 函数本身的情况下，可以通过提供不同的处理函数来改变字体的处理逻辑，体现了回调机制的灵活性和可扩展性。

再举一个例子，假设 Server 类对象中维护着多个整数，并提供了一个外部请求接口(request)。在这个例子中，该接口返回的是这些整数的求和结果。这里的求和方式可能是各数的平方和、立方和，或者是经过某种处理后的求和。也就是说，具体的求和逻辑是多变的，但我们希望保持 Server 类定义和实现的稳定性。使用回调机制时，UML 类图可如图 20-16 所示：

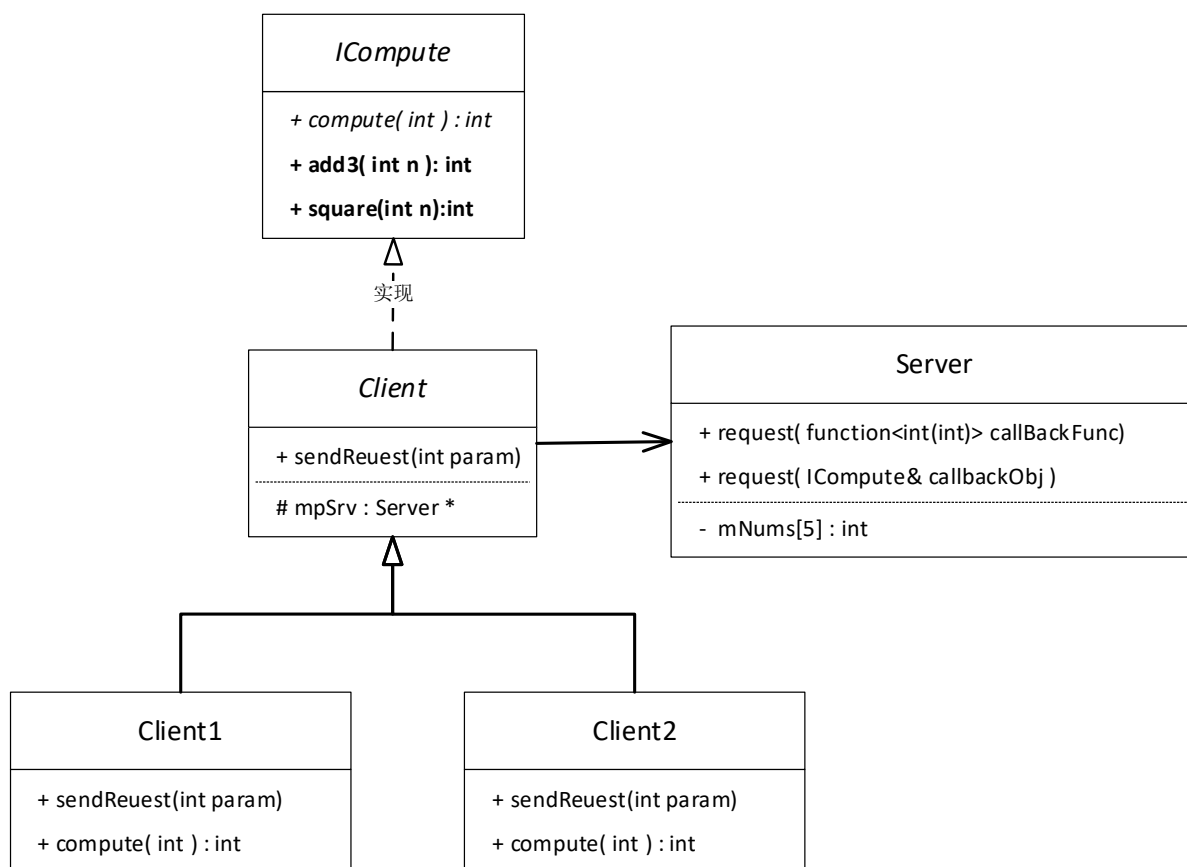


图 20-16 回调案例的 UML 类图

实现如下：

```
1 // Server.h
2 #pragma once
```

```

3  #include <functional>
4  class ICompute;
5  class Server {
6  public:
7      //以函数形式回调
8      int request( std::function < int( int ) > callbackFn );
9      //以对象形式回调
10     int request( ICompute & callbackObj, int added );
11 private:
12     int mNums[ 5 ] { 1,2,3,4,5 };
13 };

```

```

1  // Server.cpp
2  #include "Client.h"
3  #include "Server.h"
4  //以函数形式回调
5  int Server::request( std::function<int( int )> callbackFn ) {
6      int result = 0;
7      for (auto n : mNums)
8          result += callbackFn( n );
9      return result;
10 }
11 //以对象形式回调
12 int Server::request( ICompute& callbackObj, int added ) {
13     int result = added;
14     for (auto n : mNums)
15         result += callbackObj.compute( n );
16     return result;
17 }

```

```

1  // Client.h
2  #pragma once
3  class ICompute {
4  public:
5      virtual ~ICompute( ) { }
6      virtual int compute( int n ) const = 0;
7  public: //类方法
8      static int add3( int n ) { return n + 3; }    //加 3
9      static int square( int n ) { return n * n; }  //平方
10 };
11 class Server;
12 class Client : public ICompute {

```

```

13 public:
14     Client( Server& srv ) : mpSrv( &srv ) { }
15     virtual void sendRequest( int param ) = 0;
16 protected:
17     Server* mpSrv;
18 };
19 //Client 的子类 1
20 class Client1 :public Client {
21 public:
22     Client1( Server& srv ) :Client( srv ) { }
23     //演示回调的使用
24     virtual void sendRequest( int param ) override;
25     //实例方法
26     virtual int compute( int n ) const override { return 3 * n + 1; }
27 };
28 //Client 的子类 2
29 class Client2 :public Client {
30 public:
31     Client2( Server& srv, int num ) : Client( srv ), mValue( num ) { }
32     //演示回调的使用
33     virtual void sendRequest( int param ) override;
34     //实例方法
35     virtual int compute( int n ) const override { return 2 * n + mValue; }
36 private:
37     int mValue;
38 };

```

```

1 // Client.cpp
2 #include <iostream>
3 using namespace std;
4 #include "Client.h"
5 #include "Server.h"
6 int add4( int n ) { return n + 4; } //加 4
7 int cube( int n ) { return n * n * n; } //立方
8 void Client1::sendRequest( int param ) {
9     cout << "lambda 表达式作为回调函数:\n";
10    cout << mpSrv->request( [=]( int n ) { return n + param; } ) << endl;
11    cout << mpSrv->request( [=]( int n ) { return n * param; } ) << endl;
12    cout << "全局函数作为回调函数:\n";
13    cout << mpSrv->request( &add4 ) << endl;
14    cout << mpSrv->request( &cube ) << endl;
15    cout << "实例方法作为回调函数:\n"; //函数 compute 的第一个参数是 this 指针, 指定为当前对
    象的地址

```

```

16     auto func = std::bind( &Client1::compute, this, std::placeholders::_1 );
17     cout << mpSrv->request( func ) << endl;
18     cout << "类方法作为回调函数:\n";
19     cout << mpSrv->request( &Client::add3 ) << endl;
20     cout << mpSrv->request( &Client::square ) << endl;
21 }
22 void Client2::sendRequest( int param ) {
23     cout << "回调对象:\n";
24     cout << mpSrv->request( *this, param ) << endl;
25 }

```

```

1  // 主函数
2  #include "Server.h"
3  #include "Client.h"
4  int main( ) {
5      Server srv;
6      Client1 c1( srv );
7      c1.sendRequest( 3 );
8      Client2 c2( srv, 8 );
9      c2.sendRequest( 7 );
10 }

```

运行结果为:

```

lambda 表达式作为回调函数:
30
45
全局函数作为回调函数:
35
225
实例方法作为回调函数:
50
类方法作为回调函数:
30
55
回调对象:
77

```

上述代码演示了客户端通过指定不同的回调方式向服务器发送请求,从而获得不同服务结果的应用效果。在此过程中,服务器类无需进行任何修改。回调既可以是函数,也可以是对象。其中,回调函数可以采用 `lambda` 表达式、全局函数、类方法或实例方法的形式。相比回调对象,回调函数更加灵活,但在参数较多的情况下则不太适用;而回调对象由于有数据成员,能够存储更多的数据和信息,在此类场景下更具优势,此时采用回调对象的形式将更为便捷。

20.6.2 查询条件

现有代码如下：

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5  struct Book {
6      string          title; //书名
7      vector<string>  authors; //作者
8      int             pages; //页数
9  };
10 class Student {
11 public:
12     Student( ) {
13         mBooks.assign( {
14             new Book{ "面向对象程序设计",{"张三","李四" }, 300 } ,
15             new Book{ "C++面向对象程序设计",{"王五" }, 200 } ,
16             new Book{ "C++程序设计",{"赵六" }, 280 }
17         } );
18     }
19     ~Student( ) {
20         for (auto pBook : mBooks) delete pBook;
21     }
22     void readBooks( const string & partTitle ) const{           // 1
23         vector<Book *> rs {};
24         for (auto pBook : mBooks) {
25             if (titleInclude( pBook, partTitle ))
26                 rs.push_back( pBook );
27         }
28         for (auto pBook : rs) {                                   //2
29             cout << pBook->title << "\t";
30             for (auto & author : pBook->authors)
31                 cout << author << "\t";
32             cout << endl;
33         }
34     }
35     bool titleInclude( Book * p, const string & str ) const {
36         return ((p->title).find( str ) != string::npos);
37     }
38 private:
39     vector<Book *> mBooks;                                       //3
40 };
41 int main( ) {

```

```

42     Student s1;
43     s1.readBooks( "面向对象" );
44 }

```

以下是针对上述代码的新需求, 1)由于 `struct Book` 不得修改, 我们需保持其定义不变。2)禁止从 `Student` 类派生新的子类, 因此我们将通过水平关系而非继承的方式来扩展功能。3)将 //3 中的 `Book` 集合抽象到 `Library` 类中进行统一管理。相应地, `Student` 的构造和析构函数需要调整。4)为了提高移植性, 将 //2 循环中输出每本书籍内容的操作移至工具类 `Utility` 中的 `drawBook` 函数中。5)为了增强 //1 中查询条件的灵活性, 需改进设计, 允许根据不同的条件(如作者、页数、或更复杂的查询条件)进行查询。现根据上述要求, 改进设计并重新实现。

首先要统一维护 `Book` 的集合, 通常是指集合的增删改查; 其次确定 `Student` 和 `Library` 的关系, 从原始代码来看, `Student` 和 `Library` 之间采用组合或聚合关系最合适; 然后, 在工具类 `Utility` 中定义 `drawBook` 函数。这时的设计可能这样:

```

1  // Library 组合 Book
2  class Library {
3  public:
4      Library( );
5      ~Library( );
6      void addBook( const Book & book );
7      void removeBook( const std::string & title );
8      std::vector<Book *> queryBooks( ) const; //参数待定
9  private:
10     std::vector<Book *> mBooks;
11 };
12
13 //Student 聚合 Library
14 class Student {
15 public:
16     Student( Library * pLib ) :mpLib( pLib ) { }
17     void readBooks( const Condition & partTitle ) const;
18 public: //可选
19     Student( const Student & ) = delete;
20     Student & operator=( const Student & ) = delete;
21 private:
22     Library * mpLib;
23     //或 std::shared_ptr<Library> mLib; 如此不用禁止拷贝和赋值
24 };
25
26 //工具类中的方法应为类方法

```

```

27 class Utility {
28 public:
29     static void drawBook( const Book & book ) {
30         std::cout << std::setw( 32 ) << std::left << book.title;
31         for (auto & author : book.authors)
32             std::cout << std::setw( 16 ) << std::left << author;
33         std::cout << std::endl;
34     }
35 };

```

最后, 为了使查询条件更加灵活, 可以对其进行抽象和封装。具体来说, 可以将查询条件封装到一个名为 `Condition` 的类中。相应地, `titleInclude` 函数作为判断条件也应移至 `Condition` 类中, 并可更名为更具通用性的 `isTrue` 方法。当需要添加新的查询条件时, 可以通过继承 `Condition` 类创建新的子类来实现。新设计的类图如图 20-17 所示。

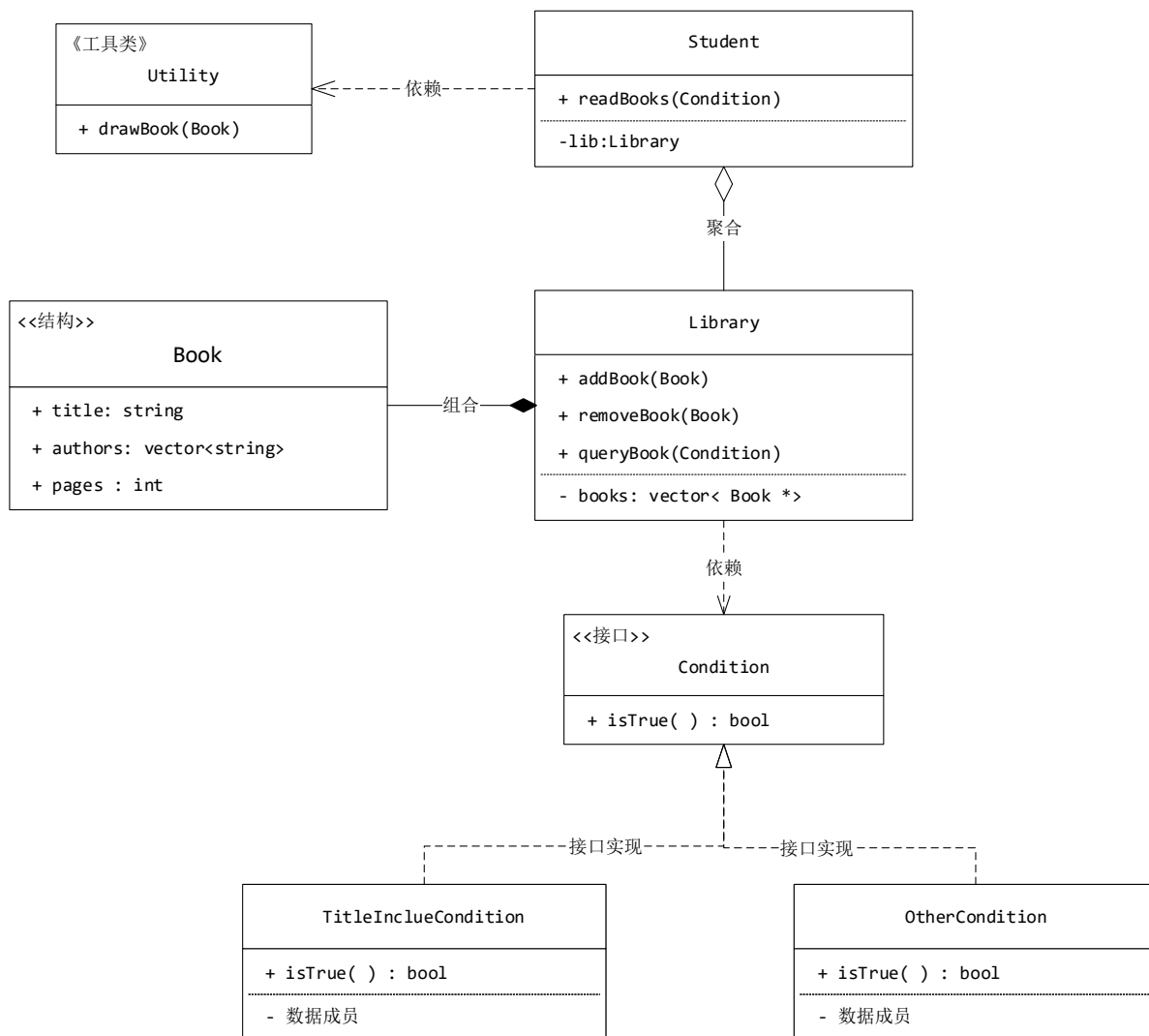


图 20-17 查询条件例的 UML 类图

具体的 Condition 类及其子类的设计可能如下:

```

1  struct Book;
2  class Condition {
3  public:
4      virtual ~Condition( ) = default;
5      virtual bool isTrue( const Book & book ) const = 0;
6  };
7
8  class TitleInclueCondition : public Condition {
9  public:
10     TitleInclueCondition( const std::string & partTitle ) : mStr( partTitle ) {}
11     virtual bool isTrue( const Book & book ) const override
12     { return ( book.title.find( mStr ) != std::string::npos ); }
13 private:
14     std::string mStr;
15 };
16
17 class OtherCondition :public Condition {
18 public:
19     virtual bool isTrue( const Book & book ) const override { /* 略 */ }
20 };

```

有了 Conditon 类及子类, Student 类及相关类的完整定义和实现, 如下:

```

1  // Library 的实现
2  Library::Library( ) {
3      mBooks.assign( {
4          new Book { "面向对象程序设计", { "张三","李四" }, 300 } ,
5          new Book { "C++面向对象程序设计", { "王五" }, 200 } ,
6          new Book { "C++程序设计", { "赵六" }, 280 }
7      } );
8  }
9
10 Library::~~Library( ) {
11     for (auto pBook : mBooks) delete pBook;
12 }
13
14 void Library::addBook( const Book & book ) {
15     mBooks.push_back( new Book( book ) );
16 }
17
18 void Library::removeBook( const std::string & title ) {
19     auto findBook = [&title]( Book * p ) { return p->title == title; };

```



```

20
21     auto it = find_if( mBooks.begin( ), mBooks.end( ), findBook );
22     if ( it != mBooks.end( ) ) {
23         delete * it;
24         auto last = std::remove_if( mBooks.begin( ), mBooks.end( ), findBook );
25         mBooks.erase( last, mBooks.end( ) );
26     }
27 }
28
29 std::vector<Book *> Library::queryBooks( const Condition & cond ) const {
30     vector<Book *> result { };
31     for (auto p : mBooks) {
32         if (cond.isTrue( *p )) result.push_back( p );
33     }
34     return result;
35 }
36
37 // Student 类的实现
38 void Student::readBooks( const Condition & cond ) const {
39     const auto & rs = mLib->queryBooks( cond );
40     for (auto p : rs)
41         Utility::drawBook( *p );
42 }
43 //主函数
44 int main( ) {
45     Library lib;
46     Student s1( &lib );
47     Student s2( &lib );
48
49     std::cout << "s1 read:\n";
50     s1.readBooks( TitleInclueCondition("面向对象") );
51     std::cout << "s2 read:\n";
52     s2.readBooks( TitleInclueCondition( "C++" ) );
53 }

```

有的同学可能会有疑问：不将条件抽象为 `Condition` 类，而是在 `readBooks` 函数中直接使用函数回调(如 `lambda` 表达式或全局条件函数)，是否也能满足需求？对于简单的条件判定，例如检查书名是否包含指定字符串，使用 `lambda` 表达式或全局函数确实可行。然而，根据需求的变化，查询条件可能变得更加复杂，此时条件函数的参数数量和类型将不再固定，导致 `readBooks` 函数的原型无法保持稳定。通过将条件抽象为 `Condition` 类，可以将复杂的条件数据存储在对象的数据成员中，从而确保 `readBooks` 函数接口的稳定性。例如，若需要提供逻辑与、逻辑或等的查询条件，可以从

Condition 类派生子类 AndCondition 和 OrCondition 类:

```

1  //逻辑与的查询条件
2  class AndCondition :public Condition {
3  public:
4      AndCondition( Condition & left, Condition & right )
5          :mpLeft( &left ), mpRight( &right ) { }
6      virtual bool isTrue( const Book & book ) const override {
7          return mpLeft->isTrue( book ) && mpRight->isTrue( book );
8      }
9  private:
10     Condition * mpLeft;
11     Condition * mpRight;
12 };
13 //逻辑或的查询条件
14 class OrCondition :public Condition {
15 public:
16     OrCondition( Condition & left, Condition & right )
17         :mpLeft( &left ), mpRight( &right ) { }
18     virtual bool isTrue( const Book & book ) const override {
19         return mpLeft->isTrue( book ) || mpRight->isTrue( book );
20     }
21 private:
22     Condition * mpLeft;
23     Condition * mpRight;
24 };

```

从中可以看出, 抽象为 Condition 类可以为查询条件提供更大的灵活性和适应性。

20.6.3 字符图像

现要定义一个字符图像类。一个字符图像由多行字符串组成, 可显示在控制台界面; 一个字符图像可以通过给字符图像加边框的方式得到, 也可以通过水平拼接两个字符图像得到, 还可以通过垂直方向上拼接两个字符图像得到。

很容易想到用二维字符数组表示字符图像, 其相应的类定义及主函数可能如下(具体实现可自行完成):

```

1  enum class Dir {
2      HORIZONTAL = 0 ,
3      VERTICAL = 1
4  };
5  class CharPic {

```

```

6  public:
7      CharPic( const char * str );          //单行字符图像
8      CharPic( const CharPic & rhs);        //拷贝构造函数
9      CharPic & operator=( const CharPic & rhs );//赋值函数
10     ~CharPic( );                          //析构函数
11     void display( ) const;                 //显示
12 public:
13     CharPic frame( ) const;                 //加框
14     CharPic hCat( const CharPic & right ) const; //水平拼接
15     CharPic vCat( const CharPic & bottom ) const; //垂直拼接
16 public: //或者使用重载的构造函数
17     CharPic( const CharPic & center );      //加框
18     //水平或垂直拼接
19     CharPic( const CharPic & pic1,const CharPic & pic2, Dir dir );
20 public: //或者使用类方法
21     //加框
22     //static CharPic frame(const CharPic & cneter );
23     //水平拼接
24     //static CharPic hCat( const CharPic & left, const CharPic & right );
25     //垂直拼接
26     //static CharPic vCat( const CharPic & left,const CharPic & bottom );
27 private:
28     int      mLines = 0; //总行数
29     std::vector<const char *> mContent; //多行字符串
30 };
31 int main( ) {
32     CharPic pic1("This is a string!");
33     CharPic pic2("This is another string!");
34     CharPic pic3( pic2 );
35     CharPic pic4(pic1, pic3 , Dir::HORIZONTAL );
36     CharPic pic5(pic3, pic4 , Dir::VERTICAL );
37     pic5.display( );
38 }

```

在上述的定义和实现中,存在一些不足:1)需要自定义拷贝、赋值函数,并且实现物理意义的深拷贝和深赋值。2)在加框、拼接的过程中,总是进行物理意义的图像复制,会降低程序效率。3)若增加新的合成方式,如对角线合成,需要修改接口。4)若修改加框的方式,如增加指定边框字符,需要修改接口。5)在构建字符图像过程中,丢失了图像的合成过程信息,也就是从合成的结果图像难以得到各子图像及合成过程。

为此,可重新设计字符图像类,核心思想是:在合成过程中,通过不同的类型记录合成过程,同时避免不必要的物理拷贝。新设计的 UML 类图如图 20-18 所示。

新设计中, 多个子类型从 CharPicImp 派生而来, 这些子类型通过聚合的方式记录合成过程中使用的子图像。这种方法不仅避免了在合成时进行物理层面的深拷贝和深赋值, 还使得扩展新的合成方式变得更加容易。例如, 若要增加对角线合成功能, 只需派生一个新的 DCatPic 类; 如需调整加框合成的参数, 也可以从 FramedPic 派生出新的子类。

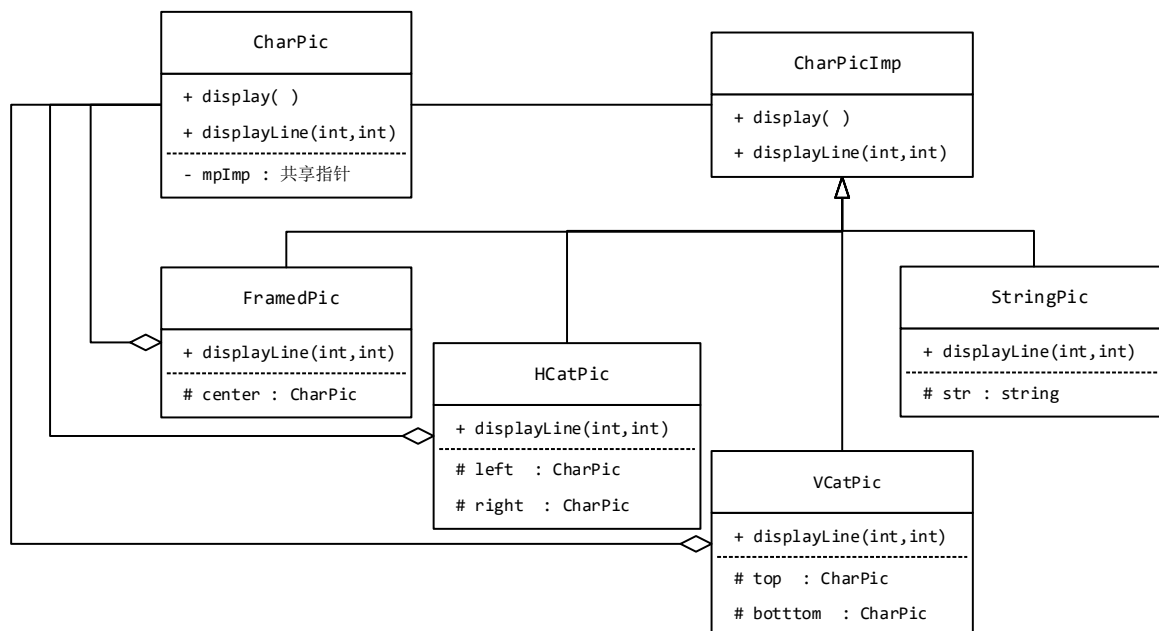


图 20-18 字符图像例的 UML 类图

在类定义与实现方面, 我们主要考虑了以下 3 点:

1. 采用 **PImp** 方法将接口与具体实现分离, 从而隐藏实现细节。这不仅便于未来扩展 **CharPicImp** 及其子类, 还确保所有 **CharPic** 类的实例对象都是“小”对象。
2. 对于 **CharPicImp** 子类中的图像数据表示, 可以选用对象型、指针型或引用型。然而, 由于引用型会禁止对象赋值操作, 在此并不推荐使用; 而指针型则可能需要自定义拷贝构造函数及赋值操作符, 并且必须保证资源释放的安全性, 虽然可以自行实现但较为复杂。因此, 本例中选择了对象型。首先, 对象型代码更为简洁; 其次, 考虑到 **CharPic** 对象本身较小, 默认的拷贝构造函数和赋值操作符执行效率是可以接受的; 最后, 为确保资源释放的安全性, **CharPic** 类中的数据成员 `mpImp` 使用了智能指针(如 `shared_ptr`)。鉴于图像合成过程不会产生自包含结构(即不存在循环引用的情况), 使用共享指针是安全的选择。
3. 关于 `display` 函数的实现, 由于字符图像通常由多行组成, 各行长度各异且内容可能来自多个子图像, 但在控制台显示时只能按行顺序依次输出。因此, 在实现过程中可能会用到一些辅助函数, 如用于逐行显示的 `displayLine()` 函数, 以及获取字符图像宽度和高度的

函数等。

以下是具体的实现代码：

```

1  // CharPic.h
2  #include <string>
3  #include <memory>
4  class CharPicImp;
5  class CharPic {
6  public: //这里直接定义成类方法，也可进一步抽象为 createPic(int type) 函数
7      static CharPic simplePic( const std::string & str );
8      static CharPic framedPic( const CharPic & center );
9      static CharPic hcatPic( const CharPic & left, const CharPic & right );
10     static CharPic vcatPic( const CharPic & top, const CharPic & bottom );
11 private:
12     CharPic( CharPicImp * pImp );
13 public:
14     void display( ) const;
15     //显示指定行，最少 columns 列，不足用空格填充
16     void displayLine( int lineIndex, int columns ) const;
17     int width( ) const;
18     int height( ) const;
19 private:
20     std::shared_ptr<CharPicImp> mpImp;
21 };

```

```

1  // CharPicImp.h
2  class CharPicImp {
3  public:
4      CharPicImp( int w = 0, int h = 0 ) :mWidth( w ), mHeight( h ) { }
5      virtual ~CharPicImp( ) { }
6      void display( ) const;
7  public:
8      virtual void displayLine( int lineIndex, int columns ) const = 0;
9      void fillPadding( int blankCount ) const;
10 public:
11     int width( ) const { return mWidth; }
12     int height( ) const { return mHeight; }
13 protected:
14     int mWidth = 0;
15     int mHeight = 0;
16 };

```

```

1  // StringPic.h
2  #pragma once
3  #include <string>
4  #include <iostream>
5  #include "CharPicImp.h"
6
7  class StringPic :public CharPicImp {
8  public:
9      inline StringPic( const std::string & str );
10 public:
11     void displayLine( int lineIndex, int columns ) const override;
12 protected:
13     const std::string mStr;
14 };
15
16 inline StringPic::StringPic( const std::string & str )
17     : CharPicImp( str.size( ), 1 ), mStr( str ) {
18 }
19 inline void StringPic::displayLine( int lineIndex, int columns ) const {
20     if ( lineIndex == 0 ) {
21         std::cout << mStr;
22         if ( mWidth < columns )
23             fillPadding( columns - mWidth ); //填充尾空格
24     } else {
25         fillPadding( mWidth );//填充空行
26     }
27 }

```

```

1  // FramedPic.h
2  #pragma once
3  #include <iostream>
4  #include "CharPic.h"
5  #include "CharPicImp.h"
6  class FramedPic : public CharPicImp {
7  public:
8      FramedPic( const CharPic & center );
9      virtual void displayLine( int lineIndex, int columns ) const override;
10 protected:
11     const CharPic mCenter;
12 };
13 inline FramedPic::FramedPic( const CharPic & center )
14     :CharPicImp( center.width( ) + 2, center.height( ) + 2 )
15     , mCenter( center ) {

```

```

16 }
17 inline void FramedPic::displayLine( int lineIndex, int columns ) const {
18     if ( lineIndex < 0 || lineIndex >= mHeight ) {
19         //超出范围
20         fillPadding( mWidth );
21     } else if ( lineIndex == 0 || lineIndex == mHeight - 1 ) {
22         //首行或尾行加框
23         for ( int c = 0; c < mWidth; c++ ) std::cout << '+';
24     } else {
25         std::cout << '+'; //首列
26         mCenter.displayLine( lineIndex - 1, columns - 2 );
27         std::cout << '+'; //尾列
28     }
29 }

```

```

1 // HCatPic.h
2 #pragma once
3 #include <iostream>
4 #include "CharPic.h"
5 #include "CharPicImp.h"
6 class HCatPic : public CharPicImp {
7 public:
8     HCatPic( const CharPic & left, const CharPic & right );
9     virtual void displayLine( int lineIndex, int columns ) const override;
10 protected:
11     const CharPic mLeft;
12     const CharPic mRight;
13 };
14
15 inline HCatPic::HCatPic( const CharPic & left, const CharPic & right )
16     :CharPicImp( left.width( ) + right.width( ),
17                 std::max( left.height( ), right.height( ) ) )
18     , mLeft( left ), mRight( right ) {
19 }
20 inline void HCatPic::displayLine( int lineIndex, int columns ) const {
21     //先左边
22     if ( lineIndex < mLeft.height( ) ) {
23         mLeft.displayLine( lineIndex, mLeft.width( ) );
24     } else {
25         fillPadding( mLeft.width( ) );
26     }
27     //再右边
28     if ( lineIndex < mRight.height( ) ) {

```

```

29     mRight.displayLine( lineIndex, mRight.width( ) );
30 } else {
31     fillPadding( mRight.width( ) );
32 }
33 }

```

```

1  // VCatPic.h
2  #pragma once
3  #include <iostream>
4  #include "CharPic.h"
5  #include "CharPicImp.h"
6  class VCatPic : public CharPicImp {
7  public:
8      VCatPic( const CharPic & top, const CharPic & bottom );
9      virtual void displayLine( int lineIndex, int columns ) const override;
10 protected:
11     const CharPic mTop;
12     const CharPic mBottom;
13 };
14
15 inline VCatPic::VCatPic( const CharPic & top, const CharPic & bottom )
16     : CharPicImp( std::max( top.width( ), bottom.width( ) ),
17         top.height( ) + bottom.height( )
18         , mTop( top ), mBottom( bottom ) {
19 }
20 inline void VCatPic::displayLine( int lineIndex, int columns ) const {
21     if ( lineIndex < mTop.height( ) ) {
22         //先上边
23         mTop.displayLine( lineIndex, columns );
24     } else if ( lineIndex >= mTop.height( ) && lineIndex < mHeight ) {
25         //下边
26         mBottom.displayLine( lineIndex - mTop.height( ), columns );
27     } else {
28         fillPadding( mWidth );
29     }
30 }

```

CharPic 类和 CharPicImp 类的实现为:

```

1  //CharPic.cpp
2  #include "CharPic.h"
3  #include "CharPicImp.h"
4  #include "StringPic.h"
5  #include "FramedPic.h"

```



```

6  #include "HCatPic.h"
7  #include "VCatPic.h"
8
9  CharPic CharPic::simplePic( const std::string & str ) {
10     return CharPic( new StringPic( str ) );
11 }
12 CharPic CharPic::framedPic( const CharPic & center ) {
13     return CharPic( new FramedPic( center ) );
14 }
15 CharPic CharPic::hcatPic( const CharPic & left, const CharPic & right ) {
16     return CharPic( new HCatPic( left, right ) );
17 }
18 CharPic CharPic::vcatPic( const CharPic & top, const CharPic & bottom ) {
19     return CharPic( new VCatPic( top, bottom ) );
20 }
21 CharPic::CharPic( CharPicImp * pImp )
22     :mpImp( pImp ) {
23 }
24 void CharPic::display( ) const {
25     mpImp->display( );
26 }
27 void CharPic::displayLine( int lineIndex, int columns ) const {
28     mpImp->displayLine( lineIndex, columns );
29 }
30 int CharPic::width( ) const { return mpImp->width( ); }
31 int CharPic::height( ) const { return mpImp->height( ); }

```

```

1  //CharPicImp.cpp
2  #include <iostream>
3  #include "CharPicImp.h"
4
4  void CharPicImp::display( ) const {
5      for ( int r = 0; r < mHeight; r++ ) {
6          displayLine( r, mWidth );
7          std::cout << '\n';
8      }
9  }
10 void CharPicImp::fillPadding( int blankCount ) const {
11     for ( int i = 0; i < blankCount; i++ )
12         std::cout << ' ';
13 }

```

最后主函数及运行结果如下:

```

1  #include "CharPic.h"
2  int main( ) {
3      CharPic pic1 = CharPic::simplePic( "This is a string!" );
4      CharPic pic2 = CharPic::simplePic( "This is a very long string!" );
5      CharPic pic3 = CharPic::vcatPic( pic1, pic2 ); //垂直拼接
6      CharPic pic4 = CharPic::framedPic( pic3 );      //加框
7      CharPic pic5 = CharPic::hcatPic( pic4, pic2 ); //水平拼接
8      CharPic pic6 = CharPic::vcatPic( pic1, pic5 ); //垂直拼接
9      CharPic pic7 = CharPic::framedPic( pic6);      //加框
10     pic7.display( );
11 }

```

运行结果为:

```

+++++
+This is a string!                      +
+++++This is a very long string!+
++This is a string!          +          +
++This is a very long string!+          +
+++++                          +

```

通过分析上述实现代码可以看出, 一个子图像可以被多次用于合成新图像, 但这个子图像只会创建了一次, 最后也只释放一次, 同时, 新的设计可以更好地适应未来可能的变化, 如水平拼接时需要增加一列中间的空列, 再比如加框时边框字符变更为'*'号等。

20.6.4 菜单系统

需求描述: 现需要开发一个基于菜单的控制台应用程序。程序执行的基本过程是: 首先显示级联菜单, 当用户选择某个菜单项后, 有两种结果: 一种是显示下级子菜单, 另一种是先执行某些业务逻辑, 然后显示处理结果, 最后显示业务菜单, 等待用户选择。执行的界面可能如下:

开始的主菜单:

```

ERP 系统主菜单
1. 系统管理
2. 组织管理
3. 计划管理
4. 项目管理
5. 生产管理
6. 市场管理
7. 财务管理
0. 退出
请选择(0-7):

```

用户选择主菜单中的"系统管理"后，显示系统管理菜单：

```

系统管理
1. 角色管理
2. 权限管理
3. 网络管理
4. 灾备管理
5. 日志管理
0. 返回
请选择(0-5):
  
```

用户选择主菜单中的"组织管理"后，显示组织管理菜单：

```

组织管理
1. 人员管理
2. 部门管理
0. 返回
请选择(0-2):
  
```

用户选择"组织管理"菜单中的人员管理后，显示人员管理菜单：

```

人员管理
1. 人员列表
2. 查询人员
3. 添加人员
4. 删除人员
5. 修改人员
0. 返回
请选择(0-5):
  
```

当用户选择"人员列表"后，会先执行业务逻辑，然后输出结果，再显示业务菜单，如：

```

                人员列表
      序号      姓名 年龄  部门      职务
      -----
      01         张三 25   项目部   经理
      02         李四 24   生产部   主管
      -----
                --- 3/4 ---
  请选择( 1.首页  2.上一页  3.下一页  4.末页  5.转指定页  6.设置页条数  0.返回 ):
  
```

当用户选择"添加人员"后，用户先输入新增人员信息，再显示业务菜单，如：

```

                添加人员
      -----
  
```

姓名： 赵虎
 年龄： 18
 部门： 市场部
 职务： 职员

 请选择(1.确定 0.取消):

上边描述了部分页面的执行情况，其它页面的情况类似。为了更好地说明面向对象设计，本案例实际上对需求做了部分简化，并主要针对菜单系统进行设计说明，没有深入讨论具体业务逻辑及相关设计。这里需要强调：在实际项目中，存在变化的地方更多，需要考虑的因素更复杂，因此真实的设计方案不会是唯一的。本案例主要解决三个问题：首先是如何设计程序的结构；其次是怎么设计相关类，使得当菜单发生变化时便于扩展；最后，如何设计使得输入输出发生变化时便于扩展。

由于在控制台界面，总是通过用户选择进行页面切换的，因此这里可将每种页面看作一种"菜单"，这样程序的执行就是在各菜单之间的不断切换。各菜单类的定义可如下：

```
1 //MenuType.h 枚举各菜单
2 enum class MenuType : unsigned long {
3     MAIN_MENU = 0,
4     //一级菜单:主菜单
5     SYSTEM_MENU = 0x1,
6     ORGANIZE_MENU = 0x2,
7     SCHEDULE_MENU = 0x3,
8     PROJECT_MENU = 0x4,
9     PRODUCE_MENU = 0x5,
10    MARKET_MENU = 0x6,
11    FINANCE_MENU = 0x7,
12    //二级菜单:系统管理
13    ROLE_MENU = 0x11,
14    AUTHORITY_MENU = 0x12,
15    NET_MENU = 0x13,
16    BACKUP_MENU = 0x14,
17    LOG_MENU = 0x15,
18    //二级菜单:组织管理
19    STAFF_MENU = 0x21,
20    DEPARTMENT_MENU = 0x22,
21    //三级菜单: 人员管理
22    STAFF_LIST_MENU = 0x211,
23    STAFF_QUERY_MENU = 0x212,
24    STAFF_APPEND_MENU = 0x213,
25    STAFF_REMOVE_MENU = 0x214,
26    STAFF_MODIFY_MENU = 0x215,
```

```

27     //其它, 略
28 };

```

```

1  //Menu.h
2  #pragma once
3  #include <iostream>
4  #include "MenuType.h"
5  //IMenu 接口类
6  class IMenu {
7  public:
8      virtual ~IMenu( ) = default;
9      virtual bool process( ) = 0;
10 };
11 //抽象类 Menu
12 class Menu :public IMenu {
13 public:
14     virtual bool process( ) override {
15         clearScreen( );          //清屏
16         doLogic( );              //处理业务逻辑及显示结果
17         display( );              //显示菜单本身
18         long choice = selectNumber( );//选择菜单项
19         return doChoice( choice );    //处理选择的菜单项
20     }
21 protected:
22     virtual void doLogic( ) { }
23     virtual void display( ) const = 0;
24     virtual long selectNumber( ) const = 0;
25     virtual bool doChoice( long choice ) const = 0;
26 protected:
27     void clearScreen( ) const;
28     long inputRangeLong( long min, long max ) const;
29 };
30 //给子类中本函数的实现提供范例
31 inline long Menu::selectNumber( ) const {
32     return inputRangeLong( 0, 0 );
33 }
34 //主菜单
35 class MainMenu :public Menu {
36 protected:
37     virtual void display( ) const override;
38     virtual long selectNumber( ) const override;
39     virtual bool doChoice( long choice ) const override;

```

```

40  };
41  //系统菜单
42  class SystemMenu :public Menu {
43  protected:
44      virtual void display( ) const override;
45      virtual long selectNumber( ) const override;
46      virtual bool doChoice( long choice ) const override;
47  };
48  //组织管理菜单
49  class OrganizeMenu :public Menu {
50  protected:
51      virtual void display( ) const override;
52      virtual long selectNumber( ) const override;
53      virtual bool doChoice( long choice ) const override;
54  };
55  //人员管理菜单
56  class StaffMenu :public Menu {
57  protected:
58      virtual void display( ) const override;
59      virtual long selectNumber( ) const override;
60      virtual bool doChoice( long choice ) const override;
61  };
62  //人员列表菜单
63  class StaffListMenu :public Menu {
64  protected:
65      virtual void doLogic( ) override;
66      virtual void display( ) const override;
67      virtual long selectNumber( ) const override;
68      virtual bool doChoice( long choice ) const override;
69  };
70  //添加人员菜单
71  class StaffAppendMenu :public Menu {
72  protected:
73      virtual void doLogic( ) override;
74      virtual void display( ) const override;
75      virtual long selectNumber( ) const override;
76      virtual bool doChoice( long choice ) const override;
77  };
78  //其它菜单, 略

```

每种菜单至多只需要一个实例, 为方便使用, 可通过 MenuMgr 类统一管理各菜单的创建、销毁以及切换。MenuMgr 类也只需要一个实例, 可定义为单例, 方便在程序中随时使用。考虑到可能有其它的单例类, 可定义单例的类模板, 简化所有单例类的定义。

```

1 //Singleton.h 定义单例的类模板
2 #pragma once
3 #include <memory>
4 template <typename T>
5 class Singleton {
6 public:
7     static T & getInstance( ) {
8         static std::shared_ptr<T> instance( new T( ) );
9         return *instance;
10    }
11 protected:
12     Singleton( ) = default;
13     Singleton( const Singleton & ) = delete;
14     Singleton( Singleton && ) = delete;
15     Singleton & operator=( const Singleton & ) = delete;
16 };

```

```

1 //具体的单例类: MenuMgr.h
2 #pragma once
3 #include <map>
4 #include "Singleton.h"
5 #include "Menu.h"
6 class MenuMgr : public Singleton< MenuMgr > {
7     friend class Singleton< MenuMgr >;
8 private:
9     MenuMgr( ); //在 mMenus 中实例化各菜单
10 public:
11     ~MenuMgr( ); //释放 mMenus 中的各菜单
12 public:
13     IMenu * activeMenu( ) const { return mActiveMenu; } //取得当前活动菜单
14     //切换菜单
15     void setActiveMenu( MenuType type ) { mActiveMenu = mMenus[ type ]; }
16 private:
17     IMenu * createMenu( MenuType type ); //实例化指定类型的 Menu
18 private:
19     IMenu * mActiveMenu;
20     using HashMap = std::map< MenuType, IMenu * >;
21     HashMap mMenus;
22 };

```

有了 MenuMgr 类，那么程序的主结构就是不断切换活动菜单的过程。

```

1 //MainERP.cpp

```

```
2  #include <memory>
3  using namespace std;
4  #include "Menu.h"
5  #include "MenuMgr.h"
6  class App {
7  public:
8      void init( );    //运行前初始化
9      void run( );     //运行
10     void term( );    //结束前清理
11 };
12 void App::init( ) { }
13 void App::run( ) {
14     bool isOver = false;
15     while ( !isOver ) {
16         IMenu * curMenu = MenuMgr::getInstance( ).activeMenu( );
17         isOver = curMenu->process( );
18     }
19 }
20 void App::term( ) { }
21
22 int main( ) {
23     std::shared_ptr<App> app = std::make_shared<App>( );
24     app->init( );
25     app->run( );
26     app->term( );
27 }
```

这样，菜单系统中主要类的 UML 类图可如图 20-19 所示。

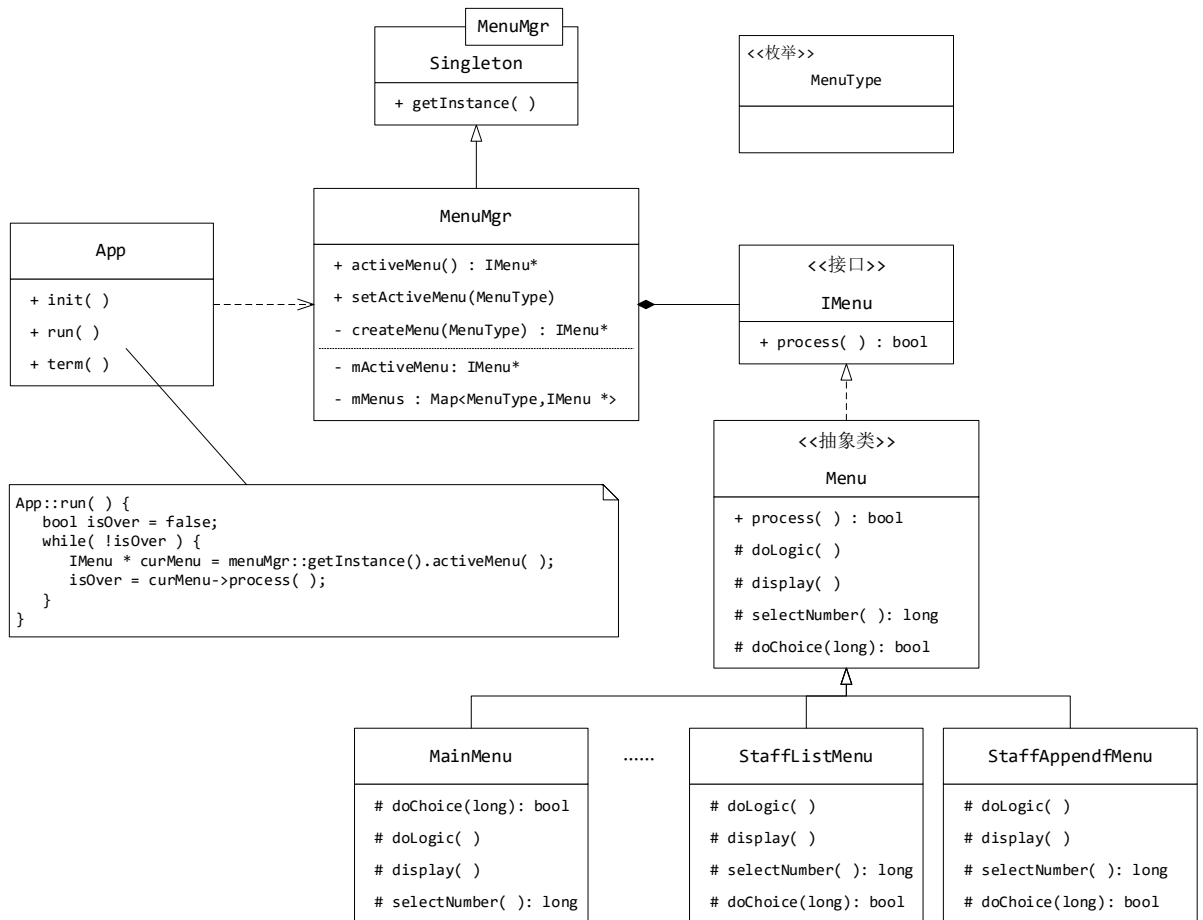


图 20-19 菜单系统的 UML 类图

下面简要说明菜单类的具体实现。对于没有业务逻辑的级联菜单，在用户选择后，可通过 MenuMgr 直接指定新的下级菜单，以实现菜单的切换。例如，抽象类 Menu 和主菜单 MainMenu:

```

1 //Menu
2 void Menu::clearScreen( ) const {
3     system( "cls" );
4 }
5 long Menu::inputRangeLong( long min, long max ) const {
6     long result = max + 1;
7     while ( result < min || result > max ) {
8         std::cin >> result;
9     }
10    return result;
11 }
12 //MainMenu
13 void MainMenu::display( ) const {
14     cout << "\n\n";
15     cout << "\tERP 管理系统\n\n";

```

```

16     cout << "\t1. 系统管理\n";
17     cout << "\t2. 组织管理\n";
18     cout << "\t3. 计划管理\n";
19     cout << "\t4. 项目管理\n";
20     cout << "\t5. 生产管理\n";
21     cout << "\t6. 市场管理\n";
22     cout << "\t7. 财务管理\n";
23     cout << "\t0. 退出\n";
24     cout << "\n\t 请选择(0-7):";
25 }
26 long MainMenu::selectNumber( ) const {
27     return inputRangeLong( 0, 7 );
28 }
29 bool MainMenu::doChoice( long choice ) const {
30     if ( choice == 0 ) {
31         return true; //退出程序
32     } else if ( choice == 1 ) {
33         //切换到系统菜单
34         MenuMgr::getInstance( ).setActiveMenu( MenuType::SYSTEM_MENU );
35     } else if ( choice == 2 ) {
36         //切换到组织菜单
37         MenuMgr::getInstance( ).setActiveMenu( MenuType::ORGANIZE_MENU );
38     } else if ( choice == 3 ) {
39         //切换到计划菜单
40         MenuMgr::getInstance( ).setActiveMenu( MenuType::SCHEDULE_MENU );
41     } else if ( choice == 4 ) {
42         //切换到项目菜单
43         MenuMgr::getInstance( ).setActiveMenu( MenuType::PROJECT_MENU );
44     } else if ( choice == 5 ) {
45         //切换到生产菜单
46         MenuMgr::getInstance( ).setActiveMenu( MenuType::PRODUCE_MENU );
47     } else if ( choice == 6 ) {
48         //切换到市场菜单
49         MenuMgr::getInstance( ).setActiveMenu( MenuType::MARKET_MENU );
50     } else if ( choice == 7 ) {
51         //切换到财务菜单
52         MenuMgr::getInstance( ).setActiveMenu( MenuType::FINANCE_MENU );
53     }
54     return false;
55 }

```

对于有业务逻辑的菜单，由于业务逻辑的多样性以及传递业务数据需要，实现方式上可有更多的选择。例如，当添加人员时，就需要记录新增人员的信息，并在用户选择"确定"后，保存该信息，

这样 `StaffAppendMenu` 类中的多个成员函数都会使用该人员信息。最简单的实现方式是，在 `StaffAppendMenu` 中定义一个人员数据成员，但这种方式会导致菜单与业务模型混在一起，增加了不必要的耦合度，最合理的方式是由业务模型维护这个人员信息。因此，可在业务模型中定义一个人员对象供各成员函数使用，作为简化，这里仍采用单例的形式：

```

1 void StaffAppendMenu::doLogic( ) {
2     Employee & one = SingleEmployee::getInstance( ).reset( ); //从业务模型中获取
3
4     cout << "\n\n\t\t 添加人员\n\n";
5     cout << "\t-----\n";
6     cout << "姓名: "; cin >> one.mName;
7     cout << "年龄: "; cin >> one.mAge;
8     cout << "部门: "; cin >> one.mDepartment;
9     cout << "职务: "; cin >> one.mPosition;
10    cout << "\t-----\n\n";
11 }
12 void StaffAppendMenu::display( ) const {
13     cout << "\n\t 确定添加吗? ( 1.确定 0.取消 ):";
14 }
15 long StaffAppendMenu::selectNumber( ) const {
16     return inputRangeLong( 0, 1 );
17 }
18 bool StaffAppendMenu::doChoice( long choice ) const {
19     if ( choice == 1 ) {
20         //仍从业务模型中获取
21         Employee & one = SingleEmployee::getInstance( ).employee( );
22         //然后，保存新增人员 one，略
23         //...
24     }
25     //返回到人员列表
26     MenuMgr::getInstance( ).setActiveMenu( MenuType::STAFF_LIST_MENU );
27     return false;
28 }

```

再比如，人员列表菜单 `StaffListMenu` 中的首页 /上一页 /下一页/末页/转指定页/设置页条数，这几个选择实际都是对分页器的处理，分页器也可以是定义为单例，如：

```

1 void StaffListMenu::doLogic( ) {
2     Pagination & pager = Pagintion::getInstance( );
3     cout << "\n\n\t\t 人员列表\n\n";
4     cout << "\t 序号\t 姓名\t 年龄\t 部门\t 职务\n";
5     //根据分页器 pager 中的分页信息,获取人员数据并显示，如下

```

```

6      cout << "\t-----\n";
7      cout << "\t01\t 张三\t25\t 项目部\t 经理\n";
8      cout << "\t02\t 李四\t24\t 生产部\t 主管\n";
9      cout << "\t-----\n\n";
10     cout << "\t          --- 3/4 ---\n\n";
11 }
12 void StaffListMenu::display( ) const {
13     cout << "\n\t 请选择";
14     cout << "( 1.首页  2.上一页  3.下一页  4.末页  5.转指定页  6.设置页条数  0.返回 ):";
15 }
16 long StaffListMenu::selectNumber( ) const {
17     return inputRangeLong( 0, 6 );
18 }
19 bool StaffListMenu::doChoice( long choice ) const {
20     if ( choice == 0 ) {
21         MenuMgr::getInstance( ).setActiveMenu( MenuType::Type::STAFF_MENU );
22         return false;
23     }
24     //取得分页器
25     Pagination & pager = Pagintaion::getInstance( );
26     if ( choice == 1 ) {
27         pager.first( );
28     } else if ( choice == 2 ) {
29         pager.prev( );
30     } else if ( choice == 3 ) {
31         pager.next( );
32     } else if ( choice == 4 ) {
33         pager.last( );
34     } else if ( choice == 5 ) {
35         int num;
36         cin>> num;
37         pager.toPage(num);
38     } else if ( choice == 6 ) {
39         int size;
40         cin >> size;
41         pager.setPageSize( size );
42     }
43     MenuMgr::getInstance( ).setActiveMenu( MenuType::Type::STAFF_LIST_MENU );
44     return false;
45 }

```

至此，系统的主要功能都具备了，但由于在代码中直接使用了 `cin` 和 `cout` 等，导致只能在控制台程序中复用各菜单类。为此，还应将输入和输出从系统中分离出来，通过解耦输入输出和各菜单

之间的联系,既可以提高菜单系统中各类的复用性,也方便未来移植到新平台或者更换输入输出方式。分离的输入输出类,可如下:

```

1  //IO.h
2  #pragma once
3  #include <string>
4  #include "Singleton.h"
5  class Input : public Singleton<Input> {
6      friend class Singleton<Input>;
7  public:
8      using ULong = unsigned long;
9      ULong  inputRangeULong( ULong min, ULong max );
10     double inputRangeDouble( double min, double max );
11     unsigned char inputChar( bool caseSensitive = false );
12     std::string inputString( );
13 private:
14     Input( ) = default;
15 };
16 class Output : public Singleton<Output> {
17     friend class Singleton<Output>;
18 public:
19     void clearScreen( );
20     void printNumber( long num );
21     void printNumberLn( long num );
22     void printDouble( double val );
23     void printDoubleLn( double val );
24     void printString( const std::string & str );
25     void printStringLn( const std::string & str );
26     void printChar( char ch );
27     void printCharLn( char ch );
28 private:
29     Output( ) = default;
30 };

```

使用 Input 类和 Outpu 类,可以将系统中的输入输出委托给 Input 类或 Output 类,例如, StaffAppendMenu::doLogic()可修改为:

```

1  void StaffAppendMenu::doLogic( ) {
2      Employee & one = SingleEmployee::getInstance( ).reset( );
3      Input & in = Input::getInstance( );
4      Output & out = Output::getInstance( );
5
6      out.printString( "\n\n\t\t 添加人员\n\n" );

```

```
7      out.println( "\t-----\n" );
8      out.println( "姓名: " ); one.mName = in.inputString( );
9      out.println( "年龄: " ); one.mAge = in.inputRangeULong(18,60);
10     out.println( "部门: " ); one.mDepartment = in.inputString( );
11     out.println( "职务: " ); one.mPosition = in.inputString( );
12     out.println( "\t-----\n\n" );
13 }
```

可自行完成程序的其它部分，包括其它类的定义和具体实现。